SIEMENS

Functions for downloading data to PLC device 1

Functions for accessing PLC service 2

TIA Portal Openness: API for automation of engineering workflows

Manual

21.00.00.00

12/2025

NOTICE

This document was downloaded from the following site:

https://docs.industrial-operations-x.siemens.cloud

Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided.

Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury **will** result if proper precautions are not taken.

WARNING

indicates that death or severe personal injury **may** result if proper precautions are not taken.

CAUTION

indicates that minor personal injury can result if proper precautions are not taken.

NOTICE

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

⚠ WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

12/2025 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1 Functions for downloading data to PLC device ... 7

1.1 Accessing system diagnostics properties for S7-1200/1500 PLCs ... 7
1.2 Downloading PC System ... 9
1.3 Downloading to PLC devices ... 13
1.4 Downloading PLC to a Windows folder ... 29
1.5 Downloading User Management data ... 33
1.6 Downloading Master Secret to PLC ... 35
1.7 Downloading to PLC with VoT ... 39
1.8 Managing PLC Master Secret in PLCs ... 41
1.9 Setting / deleting a PLC Master Secret in a PLCs ... 44
1.10 Running and stopping PLC ... 45
1.11 Supporting callbacks ... 47
1.12 Protecting PLC through password ... 48
1.13 Handling PLC block binding passwords ... 50
1.14 Uploading PLC device ... 51
1.15 Comparing PLC software ... 61
1.16 Updating PLC program ... 65
1.17 Comparing PLC hardware ... 66
1.18 Setting password policy for PLC ... 68
1.19 Managing dynamic certificate settings ... 70
1.20 Managing runtime account locking configuration ... 74

2 Functions for accessing PLC service ... 78

2.1 Access level setting ... 78
2.2 Accessing Software Checksum ... 80
2.3 Assigning PC interface ... 81
2.4 Setting PLC priority and resource assignment export ... 84
2.5 Determining the status of a PLC ... 85
2.6 Setting PLC Online ... 87
2.7 Accessing parameters of an online connection ... 89
2.8 Accessing Configuration accessible devices ... 94
2.9 Accessing fingerprint for quick station compare ... 96
2.10 Accessing Cross Reference Service on Step7 ... 107
2.11 Getting Cross References for Step7 ... 109
2.12 Setting PLC online of R/H system ... 115
2.13 Accessing software container from primary PLC of R/H system ... 117
2.14 Downloading PLCs of R/H System ... 118
2.15 Establishing or disconnecting the online connection to the PLC ... 126
2.16 Assigning project language to PLC ... 127
2.17 Assigning watch & force tables for web server and PLC display ... 128

2.18 Managing certificate 131
2.19 Managing Web Application Configuration 137
2.20 Managing system web page 137
2.21 Managing Webserver user of SIWAREX modules 139
2.22 Managing Syslog configuration 141
2.23 Supporting secure S7 communication TLS 144
2.24 Updating module description 150
2.25 Supporting UMAC on PLC device 151
2.26 Supporting IP Accessibility 157
2.27 Accessing PLC access control configuration provider 157
2.28 Setting a display password 164
2.29 Accessing web server and OPC UA user management 165

Functions for downloading data to PLC device 1

1.1 Accessing system diagnostics properties for S7-1200/1500 PLCs

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.

  See Opening a project

Introduction

You can use the TIA Portal Openness to configure system diagnostics properties for S7-1200/1500 PLC.

The following system diagnostic properties for PLC1500 are supported in TIA Portal Openness:

		• the PLC program must be recompiled • the online PLC has to be stopped to load the changed hardware configuration of the PLC • HMIs without automatic update enabled must be reloaded In the R/H system the setting of the attributes is synchronized between the two PLCs
SuppressDeactivatingSystemDiagnosticsAlarms	boolean	This attribute controls specifies whether system diagnostics alarms about the device deactivation should be suppressed, or not. This attribute is available for every PLCs of type S7-1500 with FW version 3.0 or greater, so it is not visible/available for smaller FW versions. The default value is FALSE. When the value is TRUE, the device deactivation alarms are suppressed. • Diagnostic buffer entries are not suppressed at all, therefore you can check the state of all the devices using DOE. • In the R/H system the setting of the attributes is synchronized between the two PLCs

Program code

```
Copies the following program code to clipboard.
Siemens.Engineering.HW.DeviceItem plcItem = …;
// get value of "UseFixedSystemDiagnosticsAlarmIds" via openness interfa
bool isUseFixedSystemDiagnosticsAlarmIds =
(bool)plcItem.GetAttribute("UseFixedSystemDiagnosticsAlarmIds");
// get value of "SuppressDeactivatingSystemDiagnosticsAlarms" via openne
interface
bool isSuppressDeactivatingSystemDiagnosticsAlarms =
(bool)plcItem.GetAttribute("SuppressDeactivatingSystemDiagnosticsAlarms"
// set value of "UseFixedSystemDiagnosticsAlarmIds" via openness interfa
plcItem.SetAttribute("UseFixedSystemDiagnosticsAlarmIds", false);
// set value of "SuppressDeactivatingSystemDiagnosticsAlarms" via openne
interface
plcItem.SetAttribute("SuppressDeactivatingSystemDiagnosticsAlarms", true
```

1.2 Downloading PC System

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.

  See Opening a project

Introduction

You can use the TIA Portal Openness to download PC-Station Plus and the SW-CPU independently. In order to download the PC-Station Plus configuration, you must retrieve stationmanager device item of rack of device and get the download provider as a service. SW-CPU itself must be retrieved as a device item for the same operation.

Downloading SW-CPU is similar to downloading a regular plc. For downloading SW-CPU, please see Downloading to PLC devices.

Restriction

Download of F-Activated PLCs is not currently supported for SW-CPU as well; as it is in HW-CPU.

Program code: Download software and hardware components

You are able to download software and hardware components to a device via DownloadProvider service that can be acquired from a given DeviceItem. If a DeviceItem represents a downloadable target an instance of DownloadProvider will be returned on GetService call. Otherwise it will be null.



```
DeviceItem stationManager = dev.DeviceItems.First(p =>
p.PositionNumber == 0).DeviceItems.First(a =>
a.PositionNumber == 125);

DownloadProvider downloadProviderStationManager =
stationManager.GetService<DownloadProvider>();

if (downloadProviderStationManager == null)

{

  //no download is possible for PC-Station

}

DeviceItem swCpu = dev.DeviceItems.First(p => p.Name ==
"Software PLC_1");

DownloadProvider downloadProviderSwCpu =
swCpu.GetService<DownloadProvider>();
```

Modify the following program code to configure Network parameters:



```
ConnectionConfiguration connConfig =
downloadProviderStationManager.Configuration;

ConfigurationMode configurationMode =
connConfig.Modes.Find("PN/IE");

ConfigurationPcInterface pcInterface =
configurationMode.PcInterfaces.Find("ASIX AX88179 USB 3.0 to
Gigabit Ethernet Adapter", 1);

ConfigurationTargetInterface targetInterface =
pcInterface.TargetInterfaces.Find("2 X2");

IConfiguration targetConfiguration =
pcInterface.TargetInterfaces[0];

bool isConfigured =
connConfig.ApplyConfiguration(targetInterface);

if (isConfigured)
```

```
...
```

It must be noted that, in case the target system reboots after downloading PC-Station Plus configuration, the default behavior in openness is **waiting**. This means that, the openness code flow is stopped until the first stage download is successful (and target system is rebooted) or timed out or failed.

In case you do not want to wait for the reboot, you can use a post download option which is called WaitOnReboot



```
Copies the following program code to clipboard.
//Post Download Configuration Delegate
 DownloadConfigurationDelegate postDownloadForPcStation =
downloadConfiguration =>
 {
 WaitOnReboot waitOnReboot = downloadConfiguration as
WaitOnReboot;
 if (waitOnReboot != null)
 {
 //In case user does not want to wait...
 waitOnReboot.CurrentSelection =
WaitOnRebootSelections.NoAction;
 //In case user wants to wait... This is the default option
anyway...
 //waitOnReboot.CurrentSelection =
WaitOnRebootSelections.Wait;
 return;
 }
 };
```

You must continue downloading SW-CPU separately, but it must be made sure that the download of PC-Station was executed before.



```
Copies the following program code to clipboard.
DownloadResult downloadResult = null;
try
{
//WE FIRST DOWNLOAD PC-STATION
```

```
downloadResult =
downloadProviderStationManager.Download(targetConfiguration,
preDownloadForPcStation, postDownloadForPcStation,
DownloadOptions.Hardware);
if (DownloadResultState.Error != downloadResult.State)
{
Console.WriteLine("The download is successful for pc-
station");
}
}
catch (EngineeringTargetInvocationException e)
{
Console.WriteLine("Exception Thrown, Message: " +
e.Message.ToString());
}
downloadResult = null;
try
{
downloadResult =
downloadProviderSwCpu.Download(targetConfiguration,
preDownloadForSwCpu, postDownloadForWinac,
DownloadOptions.Hardware | DownloadOptions.Software);
if (DownloadResultState.Error != downloadResult.State)
{
Console.WriteLine("The download is successful for SW-CPU");
}
}
catch (EngineeringTargetInvocationException e)
{
Console.WriteLine("Exception Thrown, Message: " +
e.Message.ToString());
 }
```

Downloading SW-CPU is similar to downloading a regular plc. For downloading SW-CPU, please see Downloading to PLC devices.

Note

Download of F-Activated PLCs is not currently supported for SW-CPU as well; as it is in HW-CPU.

A download method also triggers compilation. It is also possible to compile a device item stand alone, with the following code snippet:



```
Copies the following program code to clipboard.

ICompilable compileServiceSwCpu = swCpu.GetService<ICompilable>();

ICompilable compileServiceStationManager =
stationManager.GetService<ICompilable>();

CompilerResult compileResultStationManager =
compileServiceStationManager.Compile();

CompilerResult compileResultSwCpu =
compileServiceStationManager.Compile();

bool compileCheck =
!compileResultStationManager.State.Equals(CompilerResultState.Error);

if (compileCheck != true)

{

...

}
```

1.3 Downloading to PLC devices

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

You can use the TIA Portal Openness to download software and hardware components to PLC device through DownloadProvider (accessed from the DeviceItem). If a DeviceItem represents a downloadable target, an instance of DownloadProvider will be returned on GetService call, else the service will return null.

Program code: Retrieving DownloadProvider service from a device item

```
Copies the following program code to clipboard.
DeviceItem deviceItem = ...;
DownloadProvider downloadProvider =
deviceItem.GetService<DownloadProvider>();
if (downloadProvider != null)
{
...
}
```

In order to execute a download to a device, you will have to call Download method of DownloadProvider.

The parameters are an IConfiguration object, two delegates and DownloadOptions (Hardware, Software or Hardware and Software).

```
Copies the following program code to clipboard.
...
IConfiguration targetConfiguration = ...
DownloadConfigurationDelegate preDownloadDelegate = ...
DownloadConfigurationDelegate postDownloadDelegate = ...
DownloadOptions loadOption = DownloadOptions.Hardware |
DownloadOptions.Software;
...
DownloadResult result =
downloadProvider.Download(targetConfiguration,
preDownloadDelegate, postDownloadDelegate, loadOption);
```

An optional parameter can be used for downloading to an IP address where the IP address of the running PLC differs from the IP address configured offline.

```
Copies the following program code to clipboard.
...
ConfigurationAddress onlineAddress = ...
DownloadResult result =
downloadProvider.Download(targetConfiguration, onlineAddress,
preDownloadDelegate, postDownloadDelegate, loadOption);
```

Parameters of download method

In order to download to a PLC device, you have to call Download method of DownloadProvider. The Download method has four parameters which are IConfiguration object, two delegates and DownloadOptions (Hardware, Software or Hardware and Software).

Parameter name	Type
configuration	Siemens.Engineering.Connection.IConfiguration
onlineAddress	Siemens.Engineering.Connection.ConfigurationAddress
preDownloadConfigurationDelegate	Siemens.Engineering.Download.DownloadConfigurationDelegate
postDownloadConfigurationDelegate	Siemens.Engineering.Download.DownloadConfigurationDelegate
downloadOptions	Siemens.Engineering.Download.DownloadOptions

- Openness download is supported only if the configurations that are listed in above table are encountered and properly handled by you. If the configuration is invalid, then EngineeringTargetInvocationException is thrown and download process is aborted.
- Since compliation is a part of download, it is recommended to compile before download to get a full feedback about the complile results.

IConfiguration

You will have to provide IConfiguration object to the Download method. It will be used to establish a connection to the given PLC device. IConfiguration interface is implemented by ConfigurationAddress and ConfigurationTargetInterface. Both the objects can be accessed through ConnectionConfiguration instance. ConnectionConfiguration instance can be acquired from DownloadProvider.Connection: ConnectionConfiguration or optionally from OnlineProvider.Connection: ConnectionConfiguration properties.

Configuration of ConnectionConfiguration object is described in Accessing parameters of an online connection section.

```
Copies the following program code to clipboard.

...
DownloadProvider downloadProvider = null;
ConnectionConfiguration configuration =
downloadProvider.Configuration;
ConfigurationMode configurationMode =
configuration.Modes.Find("PN/IE");
ConfigurationPcInterface pcInterface =
configurationMode.PcInterfaces.Find("Intel(R) Ethernet
Connection I217-LM", 1);
IConfiguration targetConfiguration =
pcInterface.TargetInterfaces[0];
...
```

ConfigurationAddress

If the projected IP-address differs from the already downloaded online IP-address, an additional parameter has to use. This is for identify the correct online PLC.

```
Copies the following program code to clipboard.

 ...
DownloadProvider downloadProvider = ...
ConnectionConfiguration configuration =
downloadProvider.Configuration;
ConfigurationMode configurationMode =
configuration.Modes.Find("PN/IE");
ConfigurationPcInterface pcInterface =
configurationMode.PcInterfaces.Find("Intel(R) Ethernet
Connection I217-LM", 1);
```

```
IConfiguration targetConfiguration =
pcInterface.TargetInterfaces[0];

// Create the address object to reach the PLC online.

ConfigurationTargetInterface targetConfigurationIF =
targetConfiguration as ConfigurationTargetInterface;

ConfigurationAddressComposition downloadAddresses =
targetConfigurationIF.Addresses;

ConfigurationAddress onlineAddress =
downloadAddresses.Create("10.119.57.223");

DownloadResult result =
downloadProvider.Download(targetConfigurationIF,
onlineAddress, preDownloadDelegate, postDownloadDelegate,
loadOption);

...
```

DownloadConfigurationDelegate

You will have to provide two implementations of void DownloadConfigurationDelegate(DownloadConfiguration downloadConfiguration). First delegate will be called for pre-download configurations, and the second will be called after download is completed. The delegates will be called for each configuration that requires an action from the user. For more information about callback handling, Refer Supporting callbacks. . If configurations are not handled properly, an EngineeringTargetInvocationException will be thrown and download process will be aborted. .Certain configurations will only contain an information, therefore your action will not be required. Also, you can skip the configurations that do not prevent the download.

Note

You should avoid any modifications other than those required to resolve the download configurations in their delegate implementation.

The possible download configuration types are listed below.

Configuration name	Description and properties
DownloadConfiguration	• Base class for all the configurations. It will contain information only if it cannot be cast to a specific configuration. IConfiguration has a single property:

	• DownloadConfiguration.Message : string (read only property contains the configuration message)
DownloadSelectionConfiguration	• Base class for all configuration that can be selected. Properties of DownloadSelectionConfiguration • Does not contain additional properties. A selection must be provided in all child classes derived from it.
DownloadCheckConfiguration	• Base class for all configuration that can be checked and unchecked. • Contains single property DownloadCheckConfiguration.Checked: bool<string> (read/write property identifies whether the configuration is checked or unchecked)
DownloadPasswordConfiguration	• Base class for all configuration that required a password to proceed with download (PLC protection passwords and block binding passwords). Methods of DownloadPasswordConfiguration: • Conains a single method to set password. DownloadPasswordConfiguration.SetPassword (password: SecureString) : void - Sets password. SecureString

The datatype of the configurations are given below.

Configuration	DataType	Description and Action
DownloadConfiguration	DownloadWebApplicationBlmo	`Set Checked:bool property.` Download the WebApplication with name from Name:string
	UpdateWebApplicationBlmo	`Set Checked:bool property .` Update the WebApplication with name from

	Name:string	
DeleteWebApplicationBlmo	`Set Checked:bool property.` Delete the WebApplication with name from Name:string	

DownloadSelectionConfiguration	StartModules	`Set CurrentSelection:StartModul`
		enum values are
		`NoAction (No action)`
		`StartModule (Stop module)`
		These modules are stopped for downlo

	StopModules	`Set CurrentSelection:StopModule`
		enum values:
		`NoAction (No action)`
		`StopAll (Stop all)`
		These modules are started after the do

	AllBlocksDownload	`Set CurrentSelection:AllBlocksD`
		Available enum value is
		`DownloadAllBlocks (Download all`
		Download software to device

	OverwriteSystemData	`Set CurrentSelection:OverwriteS`
		Available enum values are
		`NoAction (No action)`
		`Overwrite (Download to device)`
		Delete and replace the existing system

	ConsistentBlocksDownload	`Set CurrentSelection:ConsistentBloc`
		Available enum value is
		`ConsistentDownload (Consistent`
		Download the software to device.

	AlarmTextLibrariesDownload	`Set CurrentSelection:AlarmTextLibra` Available enum values are

	ConsistentDownload (Consistent download) NoAction (No action) Download all alarm texts and text list texts.

ProtectionLevelChanged	Set CurrentSelection:ProtectionLevelChangedSelections. Available enum values are
	`NoChange (No change)`
	`ContinueDownloading (Continue downloading to the device)`
	CPU protection is changed to the next lower level.

ActiveTestCanBeAborted	`Set CurrentSelection:ActiveTestCanBeAbortedSelections`. Available enum values are
	`NoAction (No action)`
	`AcceptAll (Accept all)`
	Active test and commissioning functions are canceled during the loading operation of the device.

ResetModule	`Set CurrentSelection:ResetModuleSelections` Available enum values are
	`NoAction (No action)`
	`DeleteAll (Delete all)`
	It resets the module.

LoadIdentificationData	`Set CurrentSelection:LoadIdentificationDataSelections`. Available enum values are
	`LoadNothing (Load nothing)`
	`LoadData (Load data)`
	Load identification data to the PROFINET IO devices and their modules.

DifferentTargetConfiguration	`Set CurrentSelection:DifferentTargetConfigurationSelections.` Available enum values are
	`NoAction (No action)`
	`AcceptAll (Accept all)`
	Gives the difference between configured and target modules (online)

InitializeMemory	`Set CurrentSelection:InitializeMemorySelections .` Available enum values:

	`NoAction (No action)`
	`AcceptAll (Accept all)`
	This datatype is used to initialize memory.

ExpandDownload	`Set CurrentSelection: ExpandDownloadSelections`. Available enum values:
	`NoAction (No action)`
	`Download (Download)}}`
	The download must be expanded beyond your selection.

ActiveTestCanPreventDownload	`Set/Get CurrentSelection:ActiveTestCanPreventDownloadSelections`
	Available enum values:
	`NoAction (No action)`
	`AcceptAll (Accept all)`
	Active test and commissioning functions could prevent the donwload.

DataBlockReinitilization	`Set CurrentSelection:DataBlockReinitializationSelections`. Available enum values: `StopPlcAndReInitialize` (All data values, including retain data, will be initialized with their defined start values during loading. Set the PLC to STOP before loading.) `NoAction` (No action regarding "Download with reinitialization".) Different memory reserves in the online block and offline block are preventing "Download without reinitialization".
TargetForSoftware	`Set CurrentSelection:TargetForSoftwareSelections`. Available enum values: `CPU` (CPU) `PlcSimulationAdvanced` (PLC Simulation Advanced)
DownloadCheckConfiguration	CheckBeforeDownload

DownloadCheckConfiguration	UpgradeTargetDevice	`Set IsChecked:bool` property. Checks the different project versions in the configured device and target device (online).
	OverWriteHMIData	`Set IsChecked:bool` property. Overwrites if object exists online.
	FitHMIComponents	`Set IsChecked:bool` property. Components with a different version are installed on the target device.
	TurnOffSequence	`Set IsChecked:bool` property Turns off the sequence before loading.
	OverwriteTargetLanguages	`Set IsChecked:bool` property. To distinguish the settings between the project and PLC programming

DownloadCheckConfiguration	DowngradeTargetDevice	`Set IsChecked:bool` property. To mention the different data formats online and offline projects.
DownloadPasswordConfiguration	ModuleReadAccessPassword	`Set password via SetPassword(password:SecureStr` method. Enter a password to gain read access t the module.
	ModuleWriteAccessPassword	`Set password via SetPassword(password:SecureStr` method. Enter a password to gain write access the module.
	BlockBindingPassword	`Set password via SetPassword(password:SecureStr` method. Block binding password configuration method.
	PlcMasterSecretPassword	`Set password via SetPassword(password:SecureStr` method. Used for entering the PLC Master Secr password

Note

The configurations that enable to enter a password have an additional attribute

NOTICE
Please note that download configurations are similar to configurations encountered in Load preview and Load results dialogs while working with GUI of TIA Portal.

WARNING
The API user is responsible for ensuring the security measures of handling passwords through code.

IsSecureCommunication

The attribute provides 'TRUE' in the case of a "Secure communication" (TLS handshake) connection to the Plc. In all other cases the attribute delivers 'FALSE'. For more information on IsSecureCommunication, refer Supporting secure S7 communication TLS

So this concerns:

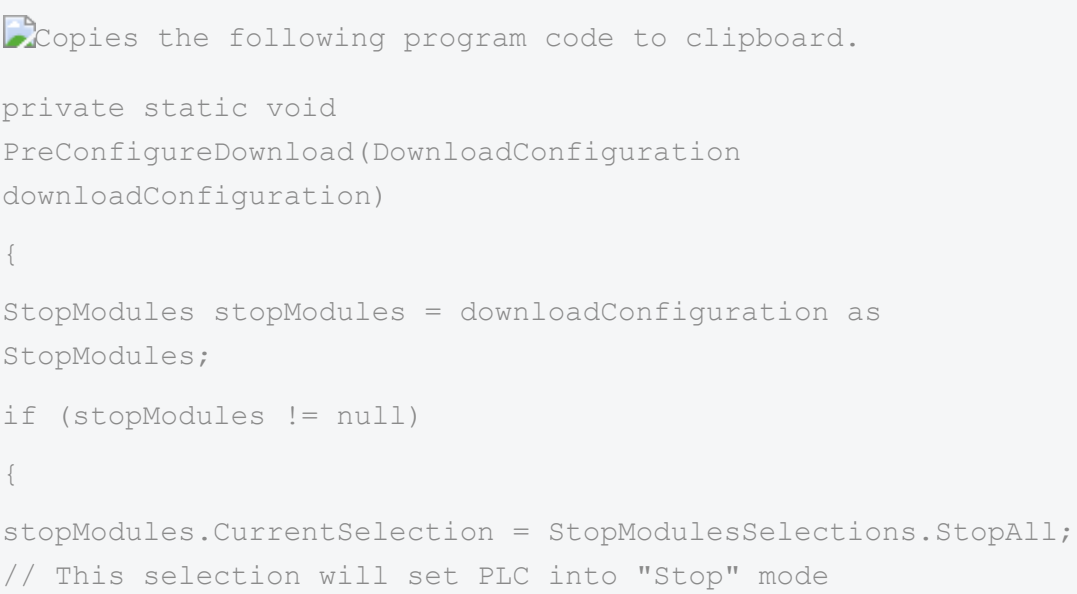
- ModuleReadAccessPassword
- ModuleWriteAccessPassword
- BlockBindingPassword
- PlcMasterSecretPassword



Unhandled exceptions inside the delegates

Unhandled configuration that can prevent the download causes an EngineeringTargetInvocationException and aborts download. An EngineeringDelegateInvocationException will be thrown in case of an unhandled exception within the Delegate.

You can use the below code example for PreDownloadDelegate implementation:



```
return;
}
AlarmTextLibrariesDownload alarmTextLibraries =
downloadConfiguration as AlarmTextLibrariesDownload;
if (alarmTextLibraries != null)
{
alarmTextLibraries.CurrentSelection =
AlarmTextLibrariesDownloadSelections.ConsistentDownload;
return;
}
BlockBindingPassword blockBindingPassword =
downloadConfiguration as BlockBindingPassword;
if(blockBindingPassword != null)
{
SecureString password = ...;// Get Binding password from a
secure location
blockBindingPassword.SetPassword(password);
return;
}
CheckBeforeDownload checkBeforeDownload =
downloadConfiguration as CheckBeforeDownload;
if (checkBeforeDownload != null)
{
checkBeforeDownload.Checked = true;
return;
}
ConsistentBlocksDownload consistentBlocksDownload =
downloadConfiguration as ConsistentBlocksDownload;
if (consistentBlocksDownload != null)
{
consistentBlocksDownload.CurrentSelection =
ConsistentBlocksDownloadSelections.ConsistentDownload;
return;
```

```
}
ModuleWriteAccessPassword moduleWriteAccessPassword =
downloadConfiguration as ModuleWriteAccessPassword;
if (moduleWriteAccessPassword != null)
{
SecureString password = ...;// Get PLC protection level
password from a secure location
moduleWriteAccessPassword.SetPassword(password);
return;
}
throw new NotSupportedException(); // Exception thrown in the
delagate will cancel download
}
```

You can use the below code example for PostDownloadDelegate implementation:

```
Copies the following program code to clipboard.
private static void
PostConfigureDownload(DownloadConfiguration
downloadConfiguration)
{
StartModules startModules = downloadConfiguration as
StartModules;
if (startModules != null)
{
startModules.CurrentSelection =
StartModulesSelections.StartModule; // Sets PLC in "Run"
mode
}
}
```

DownloadOptions

You must specify the download options through DownloadOptions flagged enum. This parameter will determine the type of download to be performed such as Hardware, Software, SoftwareOnlyChanges or a combination of them.

```
Copies the following program code to clipboard.
[Flags]
public enum DownloadOptions
{
None = 0, // Download nothing
Hardware = 1, // Download hardware. Complete download when used alone, delta download when used in combination with Software or SoftwareOnlyChanges.
Software = 2, // Download software all. Do not combine with SoftwareOnlyChanges
SoftwareOnlyChanges = 4 // Download software only changes. Do not combine with Software
}
```

Possible DownloadOptions combinations (C# code)	System Reactions
none	exception
Hardware	HW complete
Software	SW complete
Hardware | Software	HW delta, SW complete
SoftwareOnlyChanges	SW delta
Hardware | SoftwareOnlyChanges	HW delta, SW delta
Software | SoftwareOnlyChanges	exception
Hardware | Software | SoftwareOnlyChanges	exception
(DownloadOptions)11	exception

DownloadResult

The DownloadResult returned by the Download action provides feedback on the state of the objects that were downloaded.

You can use the below code example for Download invocation:



```
Copies the following program code to clipboard.
[STAThread]
static void Main()
{
...
DownloadProvider downloadProvider = ...;
IConfiguration targetConfiguration = ...;
DownloadConfigurationDelegate preDownloadDelegate =
PreConfigureDownload;
DownloadConfigurationDelegate postDownloadDelegate =
PostConfigureDownload;
DownloadResult result =
downloadProvider.Download(targetConfiguration,
preDownloadDelegate, postDownloadDelegate,
DownloadOptions.Hardware | DownloadOptions.Software);
if (result.State == DownloadResultState.Error)
{
// Handle error state
}
WriteDownloadResults(result);
...
}
private static void
PreConfigureDownload(DownloadConfiguration
downloadConfiguration)
{
...
}
private static void
PostConfigureDownload(DownloadConfiguration
downloadConfiguration)
{
```

```
...
}
private void WriteDownloadResults(DownloadResult result)
{
Console.WriteLine("State:" + result.State);
Console.WriteLine("Warning Count:" + result.WarningCount);
Console.WriteLine("Error Count:" + result.ErrorCount);
RecursivelyWriteMessages(result.Messages);
}
private void
RecursivelyWriteMessages(DownloadResultMessageComposition
messages, string indent = "")
{
indent += "\t";
foreach (DownloadResultMessage message in messages)
{
Console.WriteLine(indent + "DateTime: " + message.DateTime);
Console.WriteLine(indent + "State: " + message.State);
Console.WriteLine(indent + "Message: " + message.Message);
RecursivelyWriteMessages(message.Messages, indent);
}
}
```

1.4 Downloading PLC to a Windows folder

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.

  See Opening a project

Introduction

You can use the TIA Portal Openness to download a PLC to a Windows Folder on the Windows file system. It is possible to automate the process of creating the SIMATIC memory card, along

with the PLC program, without requiring a connection to the PLC or PLCSIM Advanced.

The feature is valid for the following products / devices:

- S7-1500 CPU including Safety (FW >= 2.1)
- S7-1500R/H CPU including Safety (HF)
- ET 200SP CPU including Safety
- ET 200pro CPU including Safety
- S7-1500 Software Controller including Safety
- ET 200SP Open Controller including Safety
- S7-1200 CPU including Safety (FW >= V4.2)

Parameter

The DownloadProvider.Download() and RHDownloadProvider.Download() accepts the following definition of parameter. Both the methods return a DownloadResult.

Parameter name	Type
directoryInfo	System.IO.DirectoryInfo
preDownloadConfigurationDelegate	Siemens.Engineering.Download.DownloadConfigurationDelegate

For more information about details of DownloadConfigurationDelegate and DownloadResult, see Downloading to PLC devices

The following configurations are supported to download PLC to Windows Folder:

Data type	Action
OverwriteOnMemoryCard	Set CurrentSelection: OverwriteOnMemoryCardSelections Available enum values: • NoAction (No action)

	• Load (Download to device)
TargetForSoftware	Set CurrentSelection: TargetForSoftwareSelections Available enum values: • CPU (CPU) • PlcSimulationAdvanced (PLC Simulation Advanced)
ConsistentBlocksDownload	Set CurrentSelection: ConsistentBlocksDownloadSelections Available enum values: ConsistentDownload (Consistent download)
AlarmTextLibrariesDownload	Set CurrentSelection: AlarmTextLibrariesDownloadSelections Available enum values: • ConsistentDownload (Consistent download) • NoAction (No action)
SafetyProgram	Set CurrentSelection: SafetyProgramSelections Available enum values: ConsistentDownload (Consistent download)

For more information about other download configuration types supported, see Downloading to PLC devices

Program code: Download To Windows Folder Specifics

Modify the following program code to access DownloadProvider for standard Plc:

```
Copies the following program code to clipboard.

DeviceItem deviceItem = ...;
DownloadProvider downloadProvider =
deviceItem.GetService<DownloadProvider>();
downloadProvider.Download(new DirectoryInfo(@"c:\tmp\card1"),
DownloadConfigurationDelegate);
```

Note

You must specify the local Windows folder on the Windows file system and a Delegate method.

Modify the following program code to access RHDownloadProvider for R/H system:

```
Copies the following program code to clipboard.
Device device = ...;
RHDownloadProvider rhDownloadProvider =
device.GetService<RHDownloadProvider>();
rhDownloadProvider.Download(new
DirectoryInfo(@"c:\tmp\card1"),
DownloadConfigurationDelegate);
```

Note

Although R/H systems are composed of two PLCs, only a single Download-Action is necessary. The saved configuration will be applied to update both the primary and backup PLC simultaneously.

For a successful download execution, the Openness program must manage the following configurations

```
Copies the following program code to clipboard.
private static void
DownloadConfigurationDelegate(DownloadConfiguration
downloadConfiguration)
{
if (downloadConfiguration is TargetForSoftware
configTargetForSoftware)
{
configTargetForSoftware.CurrentSelection
=TargetForSoftwareSelections.CPU;
}
else if (downloadConfiguration is OverwriteOnMemoryCard
configOverwriteOnMemoryCard)
{
configOverwriteOnMemoryCard.CurrentSelection =
OverwriteOnMemoryCardSelections.Load;
}
else if (downloadConfiguration is ConsistentBlocksDownload
configConsistentBlocksDownload )
```

```
{
configConsistentBlocksDownload.CurrentSelection =
ConsistentBlocksDownloadSelections.ConsistentDownload;
}
else if (downloadConfiguration is AlarmTextLibrariesDownload
configAlarmTextLibrariesDownload)
{
configAlarmTextLibrariesDownload.CurrentSelection =
AlarmTextLibrariesDownloadSelections.ConsistentDownload;
}
else if (downloadConfiguration is SafetyProgram
safetyProgram)
{
safetyProgram.CurrentSelection =
SafetyProgramSelections.ConsistentDownload;
}
}
```

Program code: DownloadResult Example

```
 Copies the following program code to clipboard.
String directory = ...;
var downloadResult = downloadProvider.Download(new
DirectoryInfo(directory), DownloadConfigurationDelegate);
PrintAllMessages(downloadResult.Messages, 0, downloadResult);
```

1.5 Downloading User Management data

Requirement

The TIA Portal Openness application is connected to the TIA Portal.
See

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Application

You can use the TIA Portal Openness to download User Management data to PLCs using UserManagementDownload class. To download User Management data you must first select the appropriate pre-download selection using UserManagementPreDownloadSelection class.

The following table provides the details of the pre-download selection available before downloading User Management data.

Enum item	Enum value	Description
KeepOnlineUserManagementData	0	UserStoreConfiguration and UserStoreAdapted are not downloaded
UpdateUserManagementDataButKeepOnlinePassword	1	Downloads complete UserStoreConfiguration and UserStoreAdapted is not deleted (i.e. keep UserStoreAdapted)
DownloadAllUserManagementDataResetToProject	2	Downloads complete UserStoreConfiguration and delete the UserStoreAdapted

Program Code: UserManagementDownload and UserManagementPreDownloadSelection

The code sample downloads the User management data with the DownloadAllUserManagementDataResetToProject pre-download selection.



```
Copies the following program code to clipboard.

var dwnlProv = plc.GetService<DownloadProvider>();
DownloadConfigurationDelegate preDwnldDelegate =
PreConfigureDownload;
DownloadConfigurationDelegate postDwnldDelegate =
PostConfigureDownload;
DownloadResult result = dwnlProv.Download(configuration:
targetConfiguration,
preDownloadConfigurationDelegate: preDwnldDelegate,
```

```
private void PreConfigureDownload(DownloadConfiguration dwnlConf)
{
if (dwnlConf is UserManagementDownload userManagementDownload)
{
userManagementDownload.CurrentSelection =
UserManagementPreDownloadSelections.DownloadAllUserManagementDataResetTo
return;
}
}
```

1.6 Downloading Master Secret to PLC

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Application

You can use the TIA Portal Openness to download Master Secrets to PLCs through PlcMasterSecretPassword accessed from download configuration. To download, you will have to provide an implementation of void DownloadConfigurationDelegate (DownloadConfiguration downloadConfiguration) for the pre-download configuration.

This delegate will be called for each configuration that requires an action from the user. The PlcMasterSecretPassword configuration handles the user action for entering the Master Secret password.

For more information about callback handling and Support of download to PLC devices, Refer Supporting callbacks and Downloading to PLC devices

Program code: PlcMasterSecretPassword Donwload

You can use the below code sample to download software and hardware components to a device via DownloadProvider service. For information on IsSecureCommunication, refer Supporting secure S7 communication TLS

```
Copies the following program code to clipboard.
internal bool DownloadDevice(DeviceItem deviceItem)
{
try
{
```

```
//Download software and hardware components to a device via
DownloadProvider service that
//can be acquired from a given DeviceItem. If a DeviceItem
represents a download able
//target an instance of DownloadProvider will be returned on
GetService call. Otherwise
//it will be null.
DownloadProvider myDownloadProvider =
deviceItem.GetService<DownloadProvider>();
if (myDownloadProvider == null)
{
return false;
}
//Preparation for IConfigurable object to establish a connection
to the device. IConfiguration
//interface is implemented by ConfigurationAddress and
ConfigurationTargetInterface. Both
//objects can be accessed via ConnectionConfiguration instance.
ConnectionConfiguration
//instance can be acquired from DownloadProvider.Configuration
ConnectionConfiguration connConfiguration =
myDownloadProvider.Configuration;
ConfigurationMode confMode =
connConfiguration.Modes.Find("PN/IE");
ConfigurationPcInterface pcInterface = ("Intel(R) Ethernet
Connection I217-LM", 1);
ConsoleOutput.PrintInterfaces(connConfiguration);
//In order to execute a download to a device, Download method of
DownloadProvider have to
//be called. The parameters are an IConfiguration object, two
delegates and DownloadOptions
// (Hardware, Software or Hardware and Software).
IConfiguration targetConfiguration =
pcInterface.TargetInterfaces[0];
```

```
DownloadConfigurationDelegate preDownloadConfigurationDelegate = PreConfigureD
DownloadConfigurationDelegate postDownloadConfigurationDelegate = PostConfigur
DownloadOptions downloadOptions = DownloadOptions.Hardware | DownloadOptions.S
// Apply the the configuration and start the download
myDownloadProvider.Configuration.ApplyConfiguration(pcInterface.TargetInterfac
var downLoadResult = myDownloadProvider.Download(targetConfiguration,
preDownloadConfigurationDelegate,
postDownloadConfigurationDelegate, downloadOptions);
return downLoadResult != null && downLoadResult.State != DownloadResultState.E
}
catch (EngineeringTargetInvocationException ex)
{
//Exception catch-ed on purpose. Client can write own code to handle the excep
//In case of PLC Master Secret exception will be thrown if:
// 1) If a different Master Secret has already been set in the PLC.
//Exception Message: An error has occurred during download: 'A different Maste
//has already been set. The PLC has to be reset!
//Possibly provide custom exception handling here
Console.WriteLine("An exception during download was thrown:");
Console.WriteLine(ex.DetailMessageData);
return false;
}
}
```

You can use the below code sample for PlcMasterSecretPassword handling in PreDownloadDelegate:

```
Copies the following program code to clipboard.
private static void PreConfigureDownload(DownloadConfiguration
downloadConfiguration)
{... ... ... ... ...
// Set the PLC Master Secret password configured in the offline
project's PLC,
// when the password was never set in the real PLC
```

```
if (downloadConfiguration is PlcMasterSecretPassword
plcMasterSecret)
{
// Get the PLC Master Secret password from a secure location
SecureString plcMasterSecretPsw = ...;
plcMasterSecret.SetPassword(plcMasterSecretPsw);
return;
}
catch (EngineeringTargetInvocationException ex)
{
// Exception catch-ed on purpose. Client can write own code to
handle the exception
// Exception will be thrown if:
// 1) plcMasterSecret password is null.
// Exception Message: The argument "password" may not be null
// 2) plcMasterSecret password is an empty string.
// Exception Message: The password provided is invalid
// 3) plcMasterSecret password is wrong.
// Exception Message: The password provided is invalid
// Possibly provide custom exception handling here
Console.WriteLine("PlcMasterSecretPassword has thrown an
exception:");
Console.WriteLine(ex.DetailMessageData);
return;
}
}
... ... ... ... ...
throw new NotSupportedException();
// Exception thrown in the delagate will cancel download
}
```

Note

An EngineeringDelegateInvocationException will be thrown in case of an unhandled exception within the Delegate.

1.7 Downloading to PLC with VoT

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness application to download, update and delete web application while downloading the relevant Plc.

You can use the DownloadConfiguration to download, update, and delete web application. For information on DownloadConfiguration, refer Downloading to PLC devices

Note

When a VoT gets newly downloaded the preDownloadConfigurationDelegate is called with the DownloadWebApplication configuration. But all further downloads of changes to this VoT will result in the UpdateWebApplication configuration.

When a VoT is already downloaded to the plc and the name of the VoT is changed, a download would result in the DownloadWebApplication configuration with the new name and the DeleteWebApplication configuration with new old name since it is considered a different VoT.

Parameter

The DownloadWebApplication, UpdateWebApplication, and DeleteWebApplication support the following parameters while using the TIA Portal Openness:

Parameter name	Data type	Description	Access
Name	string	The name of the WebApplication to download, update, and delete.	Read
Checked	boolean	The checked state of the configuration.	

		True means download	Read/Writer

Program code



```
Copies the following program code to clipboard.
using Siemens.Engineering.Download;
using Siemens.Engineering.Download.Configurations;
private static void DownloadPlcWithVOT()
{
    {
    downloadProvider.Download(targetConfiguration as
ConfigurationTargetInterface,
    PreDownloadDelegate, PostDownloadDelegate,
DownloadOptions.Software);
    }
    private static void
PreDownloadDelegate(DownloadConfiguration
downloadconfiguration)
    {
    DownloadWebApplication downloadWebApplication =
downloadconfiguration as DownloadWebApplication;
    if (downloadWebApplication != null)
    {
        if (downloadWebApplication.Name == "WebApp1")
        {
        downloadWebApplication.Checked = true;
        // This selection will download the web application
to the PLC
        }
    }
    UpdateWebApplication updateWebApplication =
downloadconfiguration as UpdateWebApplication;
    if (updateWebApplication != null)
```

```
    {
        if (updateWebApplication.Name == "WebApp2")
        {
        updateWebApplication.Checked = true;
        // This selection will update the web application on
the PLC
        }
    }
    DeleteWebApplication deleteWebApplication =
downloadconfiguration as DeleteWebApplication;
    if (deleteWebApplication != null)
    {
        if (deleteWebApplication.Name == "AppA")
        {
        deleteWebApplication.Checked = true;
        // This selection will delete the web application
from the PLC
        }
    }
    }
}
```

1.8 Managing PLC Master Secret in PLCs

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

You can use the TIA Portal Openness to protect the sensitive PLC configuration data. You can configure the PLC master secret key which will be used to encrypt the certificate configuration.

Additionally, you can disable if protecting the PLC configuration data is not a priority.

You can also use the TIA Portal Openness to configure the PLC master secret key via the HW feature PlcMasterSecretConfigurator at the device item CPU of device. The feature provides actions to protect, unprotect and change the master secret key.

- void Protect(SecureString password)
- void Unprotect(SecureString password)
- void Unprotect()
- void ChangePassword(SecureString oldPassword, SecureString newPassword)
- void ProtectAllPlcConfiguration ( )
- void UnprotectAllPlcConfiguration ( )
- void ProtectAllPlcConfigurationWithPassword(SecureString Password);
- attribute MasterSecretConfiguration MasterSecretConfiguration{get;}
- void Reset()

```
Copies the following program code to clipboard.

enum MasterSecretConfiguration
{
None = 0 , // The checkbox option “Protect the PLC Configuration data…” is unchecked.
WithPassword = 1, // The PLC Master Secret is configured.
WithoutPassword = 2 // The checkbox option “Protect the PLC Configuration data…” is
//checked and the PLC Master Secret is not configured.
WithPasswordAllDataProtection=3// The PLC Master Secret is configured and All protection enabled.
}
```

Configuring PLC master secret key

Configuring PLC master secret key is implemented as SecureString for security reasons. It is only possible to set / change the passwords . Reading the passwords is not supported.

```
Copies the following program code to clipboard.

PlcMasterSecretConfigurator plcMasterSecretConfigurator = plcCpu.GetService<PlcMasterSecretConfigurator>();
plcMasterSecretConfigurator.Protect(StringToSecureString(“TiaPassword12
plcMasterSecretConfigurator.ChangePassword(StringToSecureString(“TiaPass
StringToSecureString(“TiaPassword1234”));
```

```
public SecureString StringToSecureString(string value)
{
SecureString secureStr = new SecureString();
if (!string.IsNullOrEmpty(value))
{
for(int i = 0; i < value.Length; i++)
{
secureStr.AppendChar(value[i]);
}
}
return secureStr;
}
```

Configuring Protect all PLC configuration data, this configuration should be called once the "Protect confidential PLC configuration data" is checked either with password or without.

With password  Copies the following program code to clipboard.

```
PlcMasterSecretConfigurator plcMasterSecretConfigurator =
plcCpu.GetService<PlcMasterSecretConfigurator>();
plcMasterSecretConfigurator.ProtectAllPlcConfigurationWithPassword(new
SecureString("TiaPassword123"));
```

Without password
 Copies the following program code to clipboard.

```
PlcMasterSecretConfigurator plcMasterSecretConfigurator =
plcCpu.GetService<PlcMasterSecretConfigurator>();

plcMasterSecretConfigurator.Protect(new
SecureString("TiaPassword123"));
plcMasterSecretConfigurator.ProtectAllPlcConfiguration();
```

Disabling PLC Master Secret Functionality

Disabling PLC Master Secret password is implemented using overloaded Unprotect methods , based on the master secret configuration you can disable PLC master secret functionality with and

without master secret key. Reading the passwords is not supported.

 Copies the following program code to clipboard.

```
//Unprotect method that will be called when PlcMasterSecret
is not configured
```

```
//Unprotect method that will be called when PlcMasterSecret is configure
plcMasterSecretConfigurator.Unprotect(StringToSecureString("TiaPassword
//UnprotectAllPlcConfiguration method that will be called when PlcMaste
is configured
plcMasterSecretConfigurator.UnprotectAllPlcConfiguration();
```

Reset PLC Master Secret Functionality

If you forget master secret key , the reset will remove the Master secret configuration and the certificates will be lost.



```
Copies the following program code to clipboard.
plcMasterSecretConfigurator.Protect(StringToSecureString("TiaPassword12
plcMasterSecretConfigurator.Reset();
```

1.9 Setting / deleting a PLC Master Secret in a PLCs

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Application

You can use the TIA Portal Openness to set or reset the PlcMasterSecret to PLCs. To achieve the functionalities in the TIA Portal Openness,-OnlineProvider are extended by new Methods:

- SetPlcMasterSecret()
- ResetPlcMasterSecret()

The SetPlcMasterSecret() supports the following parameter:

Parameter	Type	Description
securePassword	SecureString	The Master Secret-password.

Note

You cannot delete or reset the Plc Master Secret by a NULL-password.

Program code: Set and Reset PlcMasterSecret

Modify the following program code to set and reset the PlcMasterSecret via the OnlineProvider service:

```
Copies the following program code to clipboard.
DeviceItem deviceItem = ...;
SecureString password = ...;
OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();
```

Modify the following program code to set the PlcMasterSecret:

```
Copies the following program code to clipboard.
DeviceItem deviceItem = ...;
SecureString password = ...;
OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();
onlineProvider.SetPlcMasterSecret(password);
```

Modify the following program code to reset the PlcMasterSecret:

```
Copies the following program code to clipboard.
DeviceItem deviceItem = ...;
OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();
onlineProvider.ResetPlcMasterSecret();
```

Note

Reset the Plc Master Secret is the same functionality like Delete in the "Online & diagnostics"-Dialog.

See also

Connecting to the TIA Portal
Opening a project

1.10 Running and stopping PLC

Requirements

- The Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- PLC is offline.

Application

When interacting with TIA Portal through Openness API, it may be necessary to change the operating mode of the PLC. TIA Portal Openness provides a way to modify the operating state of the PLC either to start or stop.

Program code

Modify the following program code for setting PLC operating state to STOP.

```
Copies the following program code to clipboard.
  public void ConfigurePreDownload(DownloadConfiguration downloadConfiguration)
   {
      StopModules stopModules = downloadConfiguration as StopModules;
      if (stopModules != null)
           {
           // Puts PLC in "Stop" mode
           stopModules.CurrentSelection = StopModulesSelections.StopAll;
      }
  }
```

Modify the following program code for setting PLC operating state to START.

```
Copies the following program code to clipboard.
 public void ConfigurePostDownload(DownloadConfiguration downloadConfiguration)
   {
    StartModules startModules = downloadConfiguration as StartModules;
```

```
       if (startModules != null)
           {
            // Puts PLC in "Start" mode
            startModules.CurrentSelection =
StartModulesSelections.StartModule;
      }
  }
```

1.11 Supporting callbacks

Requirements

- The Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

Certain API methods require an interaction with the user-defined application code during their execution. Delegates are used to handle these callback actions in the user-defined application code. You need to implement a method with a compatible signature, and pass it as a delegate parameter to the action. To proceed with the execution, TIA Portal calls the implemented methods.

Program code

```
 Copies the following program code to clipboard.

public delegate void
Siemens.Engineering.Download.DownloadConfigurationDelegate(Siemens.Engi
configuration);

...
```

Example of an user application code using and implementing the delegate:

```
 Copies the following program code to clipboard.

[STAThread]

static void Main()

{
```

```
  ...
    DownloadProvider downloadProvider = ...;
    IConfiguration targetConfiguration = ...;
    DownloadConfigurationDelegate preDownloadDelegate =
PreConfigureDownload;
    DownloadConfigurationDelegate postDownloadDelegate =
PostConfigureDownload;
    DownloadResult result =
downloadProvider.Download(targetConfiguration,
preDownloadDelegate, postDownloadDelegate,
DownloadOptions.Hardware |DownloadOptions.Software);
  ...
}
//This method will be called back by TIA Portal
private static void
ConfigurePreDownload(DownloadConfiguration
downloadConfiguration)
   {
     // Work with the parameter
   }
//This method will be called back by TIA Portal
private static void
ConfigurePostDownload(DownloadConfiguration
downloadConfiguration)
    {
      // Work with the parameter
    }
```

Note

STAThread attribute will assure that the delegates are called in the Main thread of execution.

1.12 Protecting PLC through password

Requirements

- The Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- PLC is offline.

Application

When interacting with TIA Portal through Openness API, it may be necessary to change the protection level of the PLC. TIA Portal Openness provides a way to secure the PLC through a password. The password can be set to both read-protected and write-protected PLCs.

Program code

Modify the following program code for read-protected PLCs



```
public void ConfigurePreDownload(DownloadConfiguration
downloadConfiguration)
  {
       ModuleReadAccessPassword moduleReadAccessPassword =
downloadConfiguration asModuleReadAccessPassword;
       if (moduleReadAccessPassword != null)
          {
            SecureString password = ...; // Get password from
a secure location
            moduleReadAccessPassword.SetPassword(password);
// enter the password to gain full access
          }
  }
```

Modify the following program code for write-protected PLCs



```
public void ConfigurePreDownload(DownloadConfiguration
downloadConfiguration)
  {
```

```
    ModuleWriteAccessPassword moduleWriteAccessPassword =
downloadConfigurationas ModuleWriteAccessPassword;
    if (moduleWriteAccessPassword != null)
     {
       SecureString password = ...; // Get password from a
secure location
       moduleWriteAccessPassword.SetPassword(password); //
enter the password to gain full access
     }
  }
```

WARNING

The API user is responsible for ensuring the security measures of handling passwords through code.

1.13 Handling PLC block binding passwords

Requirements

- The Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- PLC is offline.

Application

TIA Portal Openness supports the data binding of passwords for customer applications. TIA Portal Openness provides a way for the customer to specify a block binding password. For example, a block binding password can be configured on the DownloadPasswordConfiguration class by calling the SetPassword method.

Note

If you want to secure the download action with a password, a password will have to be provided during every call of download function. This is regardless of whether the device has already been configured. After the successful acceptance of password for a given configuration, all subsequent calls to SetPassword are ignored.

Program code

Modify the following program code:

```
Copies the following program code to clipboard.

public void ConfigurePreDownload(DownloadConfiguration
downloadConfiguration)
{
    DownloadPasswordConfiguration downloadPasswordConfiguration =
downloadConfiguration as DownloadPasswordConfiguration;
    if(downloadPasswordConfiguration != null &&
downloadPasswordConfiguration.Message.Contains("block_1"))
       {
              SecureString password = ...; // Get password from a
secured location
              downloadPasswordConfiguration.SetPassword(password);
    }
}
```

1.14 Uploading PLC device

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Introduction

You can use the TIA Portal Openness to upload station into a project through StationUploadProvider. The StationUploadProvider instance is returned on GetService() invoked with a project to execute an upload.

The upload station is not supported for device group.

Program code: Retrieving StationUploadProvider service from a project

```
Copies the following program code to clipboard.

Project myProject = ...;
```

```
StationUploadProvider uploadProviderForProject =
myProject.GetService<StationUploadProvider>();
if (uploadProviderForProject != null)
{
    ...
}
```

Parameters of upload method

In order to execute an upload of a PLC device, you can call StationUpload() of StationUploadProvider. The Upload method has ConfigurationAddress and UploadConfigurationDelegte parameters.

UploadOptions are optional, because the StationUpload uploads Software, Hardware, and Files.

Parameter name	Type	Description
configurationAddress	Siemens.Engineering.Connection.ConfigurationAddress	Address of device that should be uploaded
uploadConfigurationDelegate	Siemens.Engineering.Upload.UploadConfigurationDelegate	Delegate that will be called to check configuratio before upload
uploadOptions	Siemens.Engineering.Upload.UploadOptions	Upload options

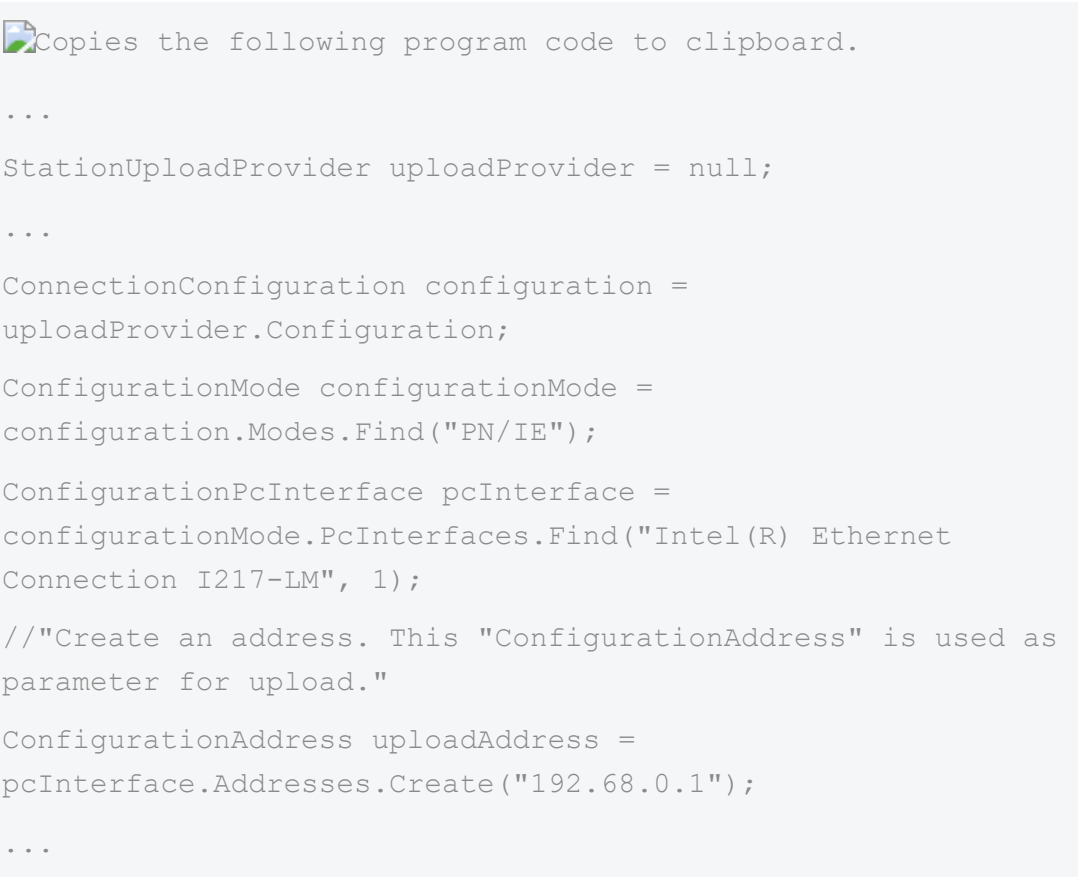
Parameter 1: ConfigurationAddress

You should provide ConfigurationAddress object to the Upload. The address object is used to establish a connection to the given PLC device that should be uploaded. The ConfigurationAddress object must be created in the ConnectionConfiguration of the StationUploadProvider.

The Configuration contains a list of supported Modes. You need to select one of the Modes that should be used for upload. The selected ConfigurationMode contains a list of all local PcInterfaces that support the selected Mode, you have to select one of the interfaces. The

desired address can be created in the Address collection of the selected ConfigurationPcInterface.

Modify the following code to create an address object:



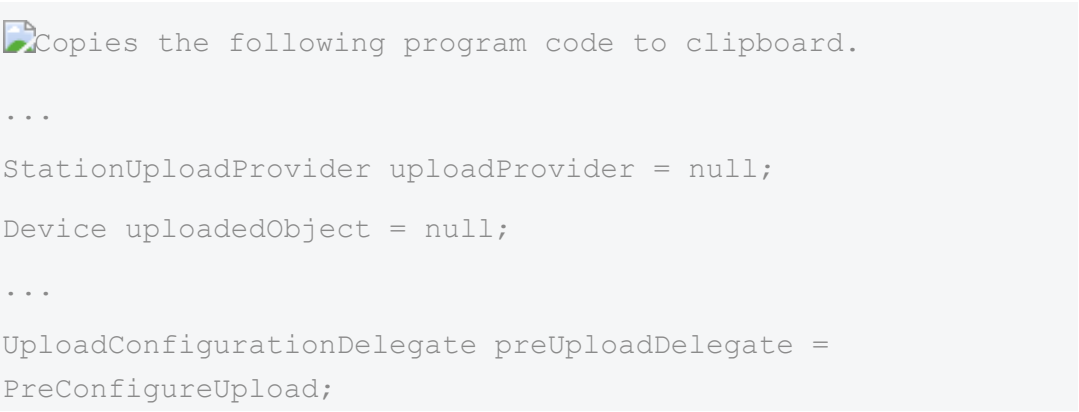
```
...
StationUploadProvider uploadProvider = null;
...
ConnectionConfiguration configuration =
uploadProvider.Configuration;
ConfigurationMode configurationMode =
configuration.Modes.Find("PN/IE");
ConfigurationPcInterface pcInterface =
configurationMode.PcInterfaces.Find("Intel(R) Ethernet
Connection I217-LM", 1);
//"Create an address. This "ConfigurationAddress" is used as
parameter for upload."
ConfigurationAddress uploadAddress =
pcInterface.Addresses.Create("192.68.0.1");
...
```

Project upload

You can start the station upload by calling the action StationUpload.

The following Parameters are mandatory:

- ConfigurationAddress: The address of the device to be uploaded
- UploadConfigurationDelegate: The callback to handle upload inhibits



```
...
StationUploadProvider uploadProvider = null;
Device uploadedObject = null;
...
UploadConfigurationDelegate preUploadDelegate =
PreConfigureUpload;
```

```
UploadResult result =
uploadProvider.StationUpload(uploadAddress,
preUploadDelegate);
 // The uploaded device
uploadedObject = result.UploadedStation;
if (uploadedObject == null)
{
 ...
}
internal void PreConfigureUpload(UploadConfiguration
uploadConfiguration)
{
 ...
}
```

Parameter2: UploadConfigurationDelegate

You need to provide an implementation of void UploadConfigurationDelegate (UploadConfiguration uploadConfiguration). The delegate will be called for pre-upload configurations. The delegate will be called for each configuration that requires an action from the user. For more information about callback handling, Refer Supporting callbacks. Certain configurations will only contain an information, therefore the user action will not be required.

The possible upload configuration types are listed below:

Configuration name	Description and properties
UploadConfiguration	• Base class for all the configurations. It contains information in the Message attribute • Contains single property UploadConfiguration.Message : string (read only property contains the configuration message)
UploadPasswordConfiguration	• Derived from UploadConfiguration • Base class for all configuration that required a password for upload. • Contains a single method to set password. UploadPasswordConfiguration.SetPassword (password: SecureString) : void - Set password

UploadSelectionConfiguration	• Derived class of UploadConfiguration • Does not contain additional properties

The datatype of the configurations are given below:

Configuration	DataType	Description and Action
UploadPasswordConfiguration	ModuleReadAccessPassword	`Set password via SetPassword(password:SecureString` Enter a password to gain read access to th
	PasswordReadAccess	`Set password via SetPassword(password:SecureString` Enter a password for SW Upload in classic access to the module.

UploadSelectionConfiguration	UploadMissingProducts	`Set CurrentSelection:UploadMissingPro`
		Available enum values:
		`TryUpload (Consistent upload)`
		`NoAction (No action)`
		Set a selection for upload.

The support of a Failsafe password is not necessary. For the read-access by uploading a F-PLC no password is needed.

IsSecureCommunication

All configurations have the attribute: IsSecureCommunication The attribute provides 'TRUE' in the case of a "Secure communication" (TLS handshake) connection to the Plc. In all other cases the attribute delivers 'FALSE'. For more information on IsSecureCommunication, refer Supporting secure S7 communication TLS

```
Copies the following program code to clipboard.
bool isSecureCommunication =
moduleReadAccessPassword.IsSecureCommunication;
if (isSecureCommunication == true)
{
// Secure communication with (Tls handshake)
...
}
```

Unhandled configuration that can prevent the upload causes an EngineeringTargetInvocationException and aborts upload.

An EngineeringDelegateInvocationException will be thrown in case of an unhandled exception within the Delegate.

```
PreUploadDelegate implementation example:
 Copies the following program code to clipboard.

private static void PreConfigureUpload(UploadConfiguration
UploadConfiguration)

{

ModuleReadAccessPassword moduleReadAccessPassword =
UploadConfiguration as ModuleReadAccessPassword;

if (moduleReadAccessPassword != null)

{

string passWD = "passWD";

var password = new SecureString();

foreach (var c in passWD)

password.AppendChar(c);

moduleReadAccessPassword.SetPassword(password);

return;

}

ModuleWriteAccessPassword moduleWriteAccessPassword =
UploadConfiguration as ModuleWriteAccessPassword;

if (moduleWriteAccessPassword != null)

{

string passWD = "passWD";

var password = new SecureString();

foreach (var c in passWD)

password.AppendChar(c);

moduleWriteAccessPassword.SetPassword(password);

return;

}

...
```

```
throw new NotSupportedException(); // Exception thrown in the
delagate will cancel upload
}
```

Parameter3: UploadOptions

You cannot specify the Upload options. This Upload options are known as: "Hardware", "Software", "Hardware and Software" and "Hardware, Software and Files".

The StationUpload works automatically with the Upload options "Hardware, Software and Files".

Upload over Router

The TIA Portal Openness supports the stationupload via Router because the station behind a router in not visibly in the project navigator. For an upload over NAT-Router this address has configured with a mapped address.

Also an upload by using a Router with activated IP-Routing is possibly.

Upload over Router with IP-Routing

The device in the network behind the router can only be reached if the router supports IP-Routing or DNAT-Routing. In difference to a NAT-Router, the upload of the PLC is also possible if the PLC is not directly accessible but via a CP or a CM.

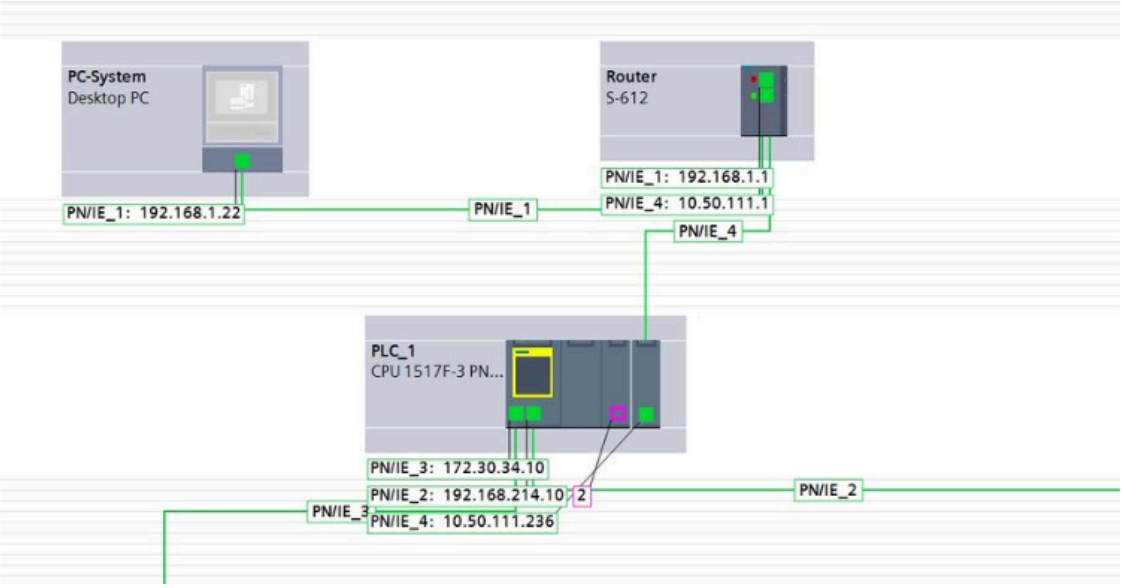
Therefore the following scenarios are supported:

- Connection between PG/PC and the PlusPLC that is directly connected to the destination subnet in combination with a router with IP-Routing.
- Connection between PG/PC and the PlusPLC that is directly connected to the destination subnet in combination with a router with DNAT-Routing
- Connection between PG/PC and PlusPLC via a communication module (CM) in combination with a router with IP-Routing
- Connection between PG/PC and PlusPLC via a communication module (CM) in combination with a router with DNAT-Routing
- Connection between PG/PC and PlusPLC via a communication processor (CP) in combination with a router with IP-Routing
- Connection between PG/PC and PlusPLC via a communication processor (CP) in combination with a router with DNAT-Routing

Note

The upload of a PLC-1200 V1 is not supported via a CM or CM; Only OMS+ protocol is supported.

For example, There is a device in a privat subnet. In this subnet the device has the address 192.168.214.10. Your PG/PC is in another subnet and has the address 192.168.1.22. Between these subnets you have a Router with IP-Routing. Now you can upload the device by using the address 10.50.111.236 of the CP.

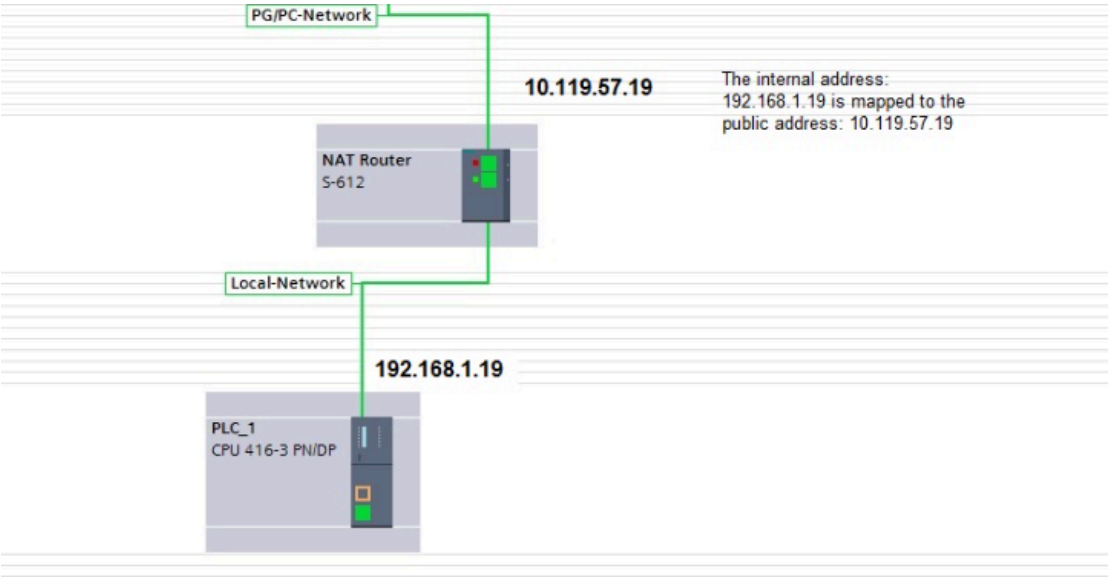


Upload over NAT-Router

A device in a private network can only be reached over a NAT Router, when the device IP-address is directly mapped to a public IP-address (destination NAT). Especially NAPT (network address and port translation) is not supported.

For example: There is a device in a privat subnet. In this subnet the device has the address 192.168.1.19. Your PG/PC is in another subnet and has the address 10.119.57.153. Between these subnets you have a NAT Router. The NAT Router needs a configuration, where the device address 192.168.1.19 is mapped to a valid address in the same subnet like the PG/PC. For example 10.119.57.19.

Now you can upload the device by using the address 10.119.57.19. (see section "**Configuration Address**").



There exist a few constrains for upload over NAT-Router.

At the first the Communication over a NAT router and S7-Routing cannot be used together. The device must be reachable directly by its IP-address from the NAT router. No S7-Router may be involved

in communication between NAT-router and device. That is the reason why the devices S71500 internal and S7300 internal cannot be reached from the PG/PC system.

The engineering system must be able to reach the NAT router directly by its IP-address. No S7-Router may be involved in communication between engineering system and NAT-router. That is the

reason while the engineering system PG/PC 2 in the above picture cannot be used for uploading a device from the Test Network.

UploadResult

The UploadResult returned by the Upload action provides feedback on the state of the objects that were uploaded.

- UploadResult.Message: UploadResultMessageComposition - Composition of UploadResultMessage

The following attributes are supported:

Attributes	Description
ErrorCount	int value of errors while upConnection between PG/PC and PlusPLC via a communication module (CM) in combination with a router with DNAT-Routingload
State	UploadResultState with possibly values: Success, Information, Warning and Error

UploadedStation	A Device-Instance of the uploaded station
WarningCount	Number of warning while upload as int

The UploadResultMessage contains:

- UploadResultMessage.Messages : UploadResultMessageComposition - Composition of UploadResultMessage

The following attributes are supported:

Attributes	Description
DateTime	System.DateTime of the created message.
ErrorCount	An int counter for errors.
State	UploadResultState with possibly values: Success, Information, Warning and Error.
WarningCount	Number of warning while upload as int

Upload invocation example

 Copies the following program code to clipboard.

```
internal bool UploadPLC()
{
...
UploadResult result =
uploadProvider.StationUpload(uploadAddress,
preUploadDelegate);
...
PrintAllMessages(result.Messages, 0);
...
}
internal void PrintAllMessages(UploadResultMessageComposition
messages, int level)
{
if (messages == null)
return;
```

```
if (level == 0)
Console.WriteLine("\n");
foreach (UploadResultMessage message in messages)
{
string messageOut =
message.Message.PadLeft(message.Message.Length + level, '\t')
+ "\n";
Console.WriteLine(messageOut);
if ((message.Messages != null) && (message.Messages.Count >
0))
PrintAllMessages(message.Messages, level+1);
}
}
```

1.15 Comparing PLC software

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Application

You have the following options to determine the deviation between the software of two devices:

- Comparing the software of two configured PLCs
- Comparison of the software of a PLC and the project library
- Comparison of the software of a PLC and the global library
- Comparison of the software of a PLC and the master copy of a PLC
- Comparisonof the software of a configured PLC with the software of a connected PLC in "Online" status

Signature

Use the `CompareTo` or `CompareToOnline` methods for the comparison.

```
public CompareResult CompareTo (ISoftwareCompareTarget compareTarget)
```

```
public CompareResult CompareToOnline ()
```

Return value / parameter	Function
`CompareResult compareResult`	Returns the comparison result: • `FolderContentsDifferent`: Content of the compared folders differs. • `FolderContentsIdentical`: Content of the compared folders is identical. • `ObjectsDifferent`: Content of the compared objects differs. • `ObjectsIdentical`: Content of the compared objects is identical. • `LeftMissing`: The object is not contained in the object from which the comparison was started. • `RightMissing`: The object is not contained in the object which is being compared. • `CompareIrrelevant`: Comparison of these 2 objects is irrelevant • `FolderContainsDifferencesOwnStateDifferent`: Folder contents have one or more differences, folder's own state different • `FolderContentEqualOwnStateDifferent`: Folder content is the same, folder's own state is different
`ISoftwareCompareTarget compareTarget`	List of comparable objects.

Program code

Modify the following program code to output the comparison result:

```
Copies the following program code to clipboard.

private static void WriteResult(CompareResultElement
compareResultElement, string indent)
{
    Console.WriteLine("{0} <{1}> <{2}> <{3}> <{4}> ",
    indent,
```

```
    compareResultElement.LeftName,
    compareResultElement.ComparisonResult,
    compareResultElement.RightName,
    compareResultElement.DetailedInformation);
    WriteResult(compareResultElement.Elements, indent);
}
private static void WriteResult
(IEnumerable<CompareResultElement> compareResultElements,
string indent)
{
    indent += " ";
    foreach (CompareResultElement compareResultElement in
compareResultElements)
    {
        WriteResult(compareResultElement, indent);
    }
}
```

Modify the following program code to compare the software of devices:

```
Copies the following program code to clipboard.
private static void CompareTwoOfflinePlcs(PlcSoftware
plcSoftware0, PlcSoftware plcSoftware1)
{
    if (plcSoftware0 != null && plcSoftware1 != null)
    {
        CompareResult compareResult =
plcSoftware0.CompareTo(plcSoftware1);
        WriteResult(compareResult.RootElement, string.Empty);
    }
}
```

Modify the following program code to compare the software of a PLC with the project library:

```
Copies the following program code to clipboard.
```

```
private static void ComparePlcToProjectLibrary(Project
project, PlcSoftware plcSoftware)
{
    if (project != null && plcSoftware != null)
    {
        CompareResult compareResult =
plcSoftware.CompareTo(project.ProjectLibrary);
        WriteResult(compareResult.RootElement, string.Empty);
    }
}
```

Modify the following program code to compare the software of a PLC with the global library:

```
private static void ComparePlcToGlobalLibrary(PlcSoftware
plcSoftware, GlobalLibrary globalLibrary)
{
    if (plcSoftware != null && globalLibrary != null)
    {
        CompareResult compareResult =
plcSoftware.CompareTo(globalLibrary);
        WriteResult(compareResult.RootElement, String.Empty);
    }
}
```

Modify the following program code to compare the software of a PLC with a master copy:

```
private static void ComparePlcToMasterCopy(Project project, PlcSoftware
plcSoftware)
{
    if (project != null && plcSoftware != null)
    {
        CompareResult compareResult =
plcSoftware.CompareTo(project.ProjectLibrary.MasterCopyFolder.MasterCopi
        WriteResult(compareResult.RootElement, string.Empty);
```

```
    }
}
```

Modify the following program code to compare the software of a PLC with the software of a connected PLC:

```
private static void ComparePlcToOnlinePlc(PlcSoftware
plcSoftware)
{
    if (plcSoftware != null)
    {
        CompareResult compareResult =
plcSoftware.CompareToOnline();
        WriteResult(compareResult.RootElement, string.Empty);
    }
}
```

1.16 Updating PLC program

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.

  See Opening a project

Introduction

You can use the UpdateProgram() to upgrade the instruction to the latest version of PLC. You can access the UpdateProgram() through PlcSoftware instance.

Program code

```
private void UpdateProgramPLC()
{
PlcSoftware plcSoftware = ...;
```

```
plcSoftware.UpdateProgram();
}
```

See also

Opening a project

1.17 Comparing PLC hardware

Requirements

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Application

You can use the TIA Portal Openness to compare the hardware of two PLC devices.

Signature

Use the CompareTo method for the comparison of two hardware objects.

CompareResult CompareTo (IHardwareCompareTarget compareTarget);

Return value/parameter	Function
CompareResult compare result	Return the comparision result: • FolderContainsDifferencesOwnStateDifferent: Folder contents have one or more differences, folder's own state is different • FolderContentEqualOwnStateDifferent: Folder content is the same, folder's own state is different.
IHardwareCompareTarget compareTarget	The compare target for which the hardware compare should be performed. Must not be null.

If the Parameter compareTarget is null and an attempt is made to compare the hardware will throw Siemens.Enginnering.EngineeringTargetInvocationExceptions.

Program

Modify the following program code to output the comparision result:

```
Copies the following program code to clipboard.

...
CompareResult compareResult = plc_1.CompareTo(plc_2);
CompareResultState resultState =
compareResult.RootElement.ComparisonResult;
if (resultState ==
CompareResultState.FolderContainsDifferencesOwnStateDifferent)
{
 // Folder contents have one or more differences, folder's own
state is different:
 // May occur if the plc has a different subordinate element,
e.g., a local module, and
 // the plc itself is different, e.g., in a parameter
}
else if (resultState ==
CompareResultState.FolderContentEqualOwnStateDifferent)
{
 // Folder content is the same, folder's own state is
different:
 // May occur if a folder-style module, e.g., FM 351, has
equal subordinate elements but
 // the module itself is different, e.g., in a parameter
}
else if (resultState ==
CompareResultState.FolderContentsIdentical)
{
 ...
}
...
```

See also

Connecting to the TIA Portal
Opening a project

1.18 Setting password policy for PLC

Requirement

- The TIA Portal is connected to the TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to set the security password policy for legacy and S7-1200/1500 PLC.

Setting password policy for S7-1200/1500 PLC

You can use the PlcPasswordPolicyService at the project base object to set the security password policy for S7-1200/1500 PLC.

The PlcPasswordPolicyService feature supports the following attributes:

Attribute name	Data type	Description
PasswordPolicyEnabled	Boolean	If PasswordPolicyEnabled is "True" then the UMAC (i.e. Password complexity for project users) configured attribute values will be considered for validating the password If PasswordPolicyEnabled is "False" then new policy cannot be applied for any S7-1200/1500 PLC.

See Setting password policies for UMAC for more information.

Setting password policy for legacy PLC(S7-300/400, ET 200 S PLCs, WinAC)

You can use the LegacyPlcPasswordPolicyService at the project base object to set the security password policy for legacy PLC.

The LegacyPlcPasswordPolicyService feature supports the following attributes:

Attribute name	Data type	Description
PasswordPolicyEnabled	Boolean	If PasswordPolicyEnabled is "True" then all the other configured attribute values will be

		considered for validating the password. If PasswordPolicyEnabled is "False" then new policy cannot be applied for any legacy plc.
MinimumLength	Short	Sets the value of minimumLength of the password The allowed range value for minimum length is 5 to 8
MinimumNumericCharacterLength	Short	Sets the value of minimumNumericCharacterLength in password The allowed range value for minimum numeric character length is 0 to 8
MinimumSpecialCharacterLength	Short	Sets the value of minimumSpecialCharacterLength in password The allowed range value for minimum numeric character length is 0 to 8
IncludesLowerCaseAndUpperCaseCharacters	Boolean	Sets the value of includesLowerCaseAndUpperCaseCharacters in password

A custom exception PasswordPolicySettingsException is thrown when you attempt to set the attribute value outside the permissible range.

Exception

When the password is set from the Openness script for a protection access level, a new type of exception EngineeringPasswordPolicyViolationException will be thrown if the entered password does not comply with the policy.

Program code

To set the password policies for legacy PLC modify the following program code:

```
Copies the following program code to clipboard.

public void
LegacyPlc_SecuritySettingPasswordPolicy_SetAttributes()

{
```

```
m_LegacyPlcPasswordPolicyService.PasswordPolicyEnabled= true;
m_LegacyPlcPasswordPolicyService.MinimumLength = 6;
m_LegacyPlcPasswordPolicyService.MinimumNumericCharacterLength = 2;
m_LegacyPlcPasswordPolicyService.MinimumSpecialCharacterLength=2;
m_LegacyPlcPasswordPolicyService.IncludeLowerCaseAndUpperCaseCharacters
= true;
}
```

To set the password policies for S7-1200/1500 PLC modify the following program code:

```
Copies the following program code to clipboard.
public void
PlusPlc_SecuritySettingPasswordPolicy_SetAttributes()
{
PlcPasswordPolicyService m_PlcPasswordPolicyService =
m_project.GetService<PlcPasswordPolicyService>();
m_PlusPlcPasswordPolicyService.PasswordPolicyEnabled= true;
}
```

1.19 Managing dynamic certificate settings

Requirement

- The TIA Portal is connected to the TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to configure dynamic certificate management for all supported services at a central place within the TIA Portal instead of modifying each application and service individually.

You can use the Certificate Management Configuration service to manage dynamic certificate, which will be available for the PLC instance of all PLCs device family with version V3.0 and later.

Property

The CertificateManagementConfiguration supports the following properties:

Property name	Data type	Description	Access	Note
Usage	CertificateConfigurationUsage	Gets or sets the usage of the certificate management configuration The Usage property allows you to choose two options either certificates configured and download using TIA Portal or certificates provided by certificate management during runtime . The Usage property has two enum values: • TIAPortal • Runtime	Read/Write	The Engi is th valu Cert whe in de
CertificateExpirationEventActivated	Boolean	Enable or disable the certificate expiration setting of the certificate management configuration	Read/Write	The Engi is th as ce dow

RemainingCertificateLifetime	ushort	Gets or sets the value for showing event for remaining certificate life time of the certificate management configuration This is a percentage value that allows to calculate the time after which a PLC alarm shall be raised to inform the user about an upcoming expiration.	Read/Write
CertificateSupportedServices	CertificateSupportedServiceComposition	Gets all the certificate supported services of the certificate management configuration which enabled dynamic certificate management. The services currently supported are OPC UA and Webserver.	

		The services currently supported are OPC UA and Webserver.	Read	

Method

The CertificateSupportedService() represents the supported services of the Certificate Management Configuration. It accepts the following property:

Property name	Data type	Description	Access
ServiceType	CertificateSupportedServiceName	Represents name of the service supported by certificate management configuration. The CertificateSupportedServiceName accepts the following Enum item and its Enum value: • Webserver - 100 • OpcUaServer - 200	Read
Id	ushort	Represents id of the service supported by certificate management configuration	Read
ServiceGroupName	string	Represents group name of supported services of the certificate management configuration	Read/Write

Program code



```
Copies the following program code to clipboard.
//...
// for getting the service
CertificateManagementConfiguration
certifcateConfigurationService =
plc.GetService<CertificateManagementConfiguration>();
// To setup the usage
certifcateConfigurationService.Usage =
CertificateConfigurationUsage.TIAPortal;
```

```
var servicesCount =
certifcateConfigurationService.CertificateSupportedServices.Count;
// For setting group name
certifcateConfigurationService.CertificateSupportedServices[0].ServiceG
= "ServiceName_1";
```

Handle dynamic certificates during download to PLC

The SelectiveDeleteDownload can be used to handle certificates during download to PLC. Based on this option, dynamic certificates will be deleted.

The SelectiveDeleteDownload.CurrentSelection supports the following Enum item and its associated enum value:

- SelectiveDeleteDataSelections.AcceptAll - accept all
- SelectiveDeleteDataSelections.DeleteSelected - delete selected
- SelectiveDeleteDataSelections.DeleteAll - delete all

Program code: Handle dynamic certificates during download to PLC

 Copies the following program code to clipboard.

```
private void
DownloadConfigurationDelegate(DownloadConfiguration
downloadConfiguration)
{
    if (downloadConfiguration is SelectiveDeleteDownload
selectiveDeleteDownload)
    {
     selectiveDeleteDownload.CurrentSelection =
SelectiveDeleteDataSelections.AcceptAll;
    }
}
```

1.20 Managing runtime account locking configuration

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

See Opening a project

Introduction

You can use the TIA Portal Openness to configure enable/disable Account locking at runtime. You also have the option to configure different values for account locking at runtime properties. This configuration can be achieved through TIA Portal or by Runtime management.

The TIA Portal Openness provides PlcAccountLockingAtRuntimeFeature service that provides you information about the account locking at runtime configuration and ways to modify the configuration.

The PlcAccountLockingAtRuntimeFeature service provides the following properties to configure the account locking at runtime:

Property name	Data type	Description	Access	Note
EnableLockUserAccountAtRuntime	Boolean	Enable account locking at runtime feature with default values	Read/Write	The default value for property is True
NumberOfFailedLoginAttempts	UInt32	Specifies the number of failed login attempts	Read/Write	The default value for property is 5 and range is [1-99]
LoginRetryInterval	UInt32	Specifies the login retry interval in minutes	Read/Write	The default value for propety is 2 and range is [1-30]
LockAccountForDuration	UInt32	Specifies the duration		

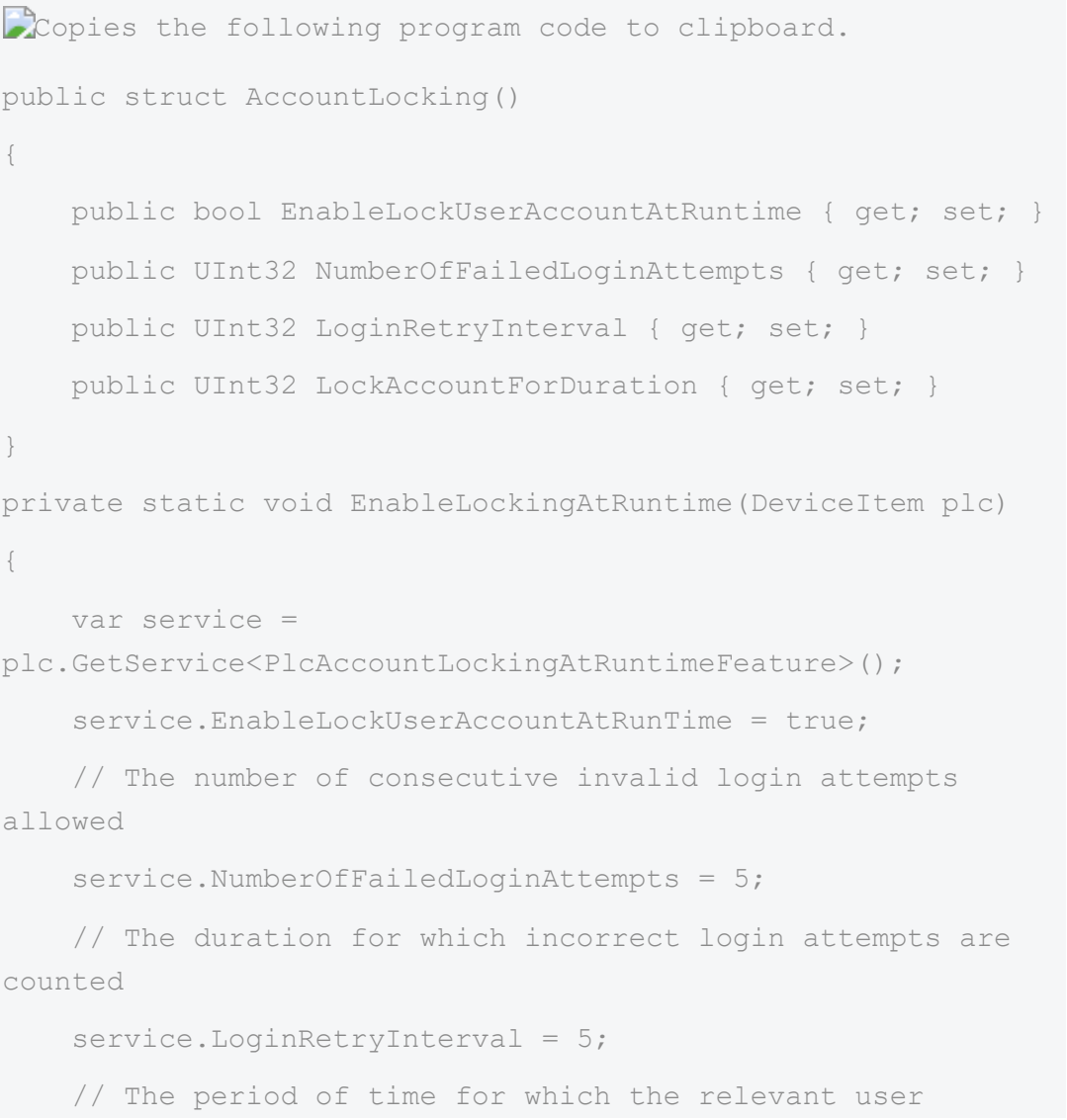
		for account lock	Read/Write	The default value for propety is 2 and range is [1-60]

Note

- The values of the properties are downloaded only when the local user management option is activated in Access Control.
- This feature is available in the following PLC versions only.
  - S7-1500 V4.1 and above
  - S7-1200 G2 V4.1 and SpeedBoat V4.1

Program code

To set and get the account locking at runtime, modify the following program code:



```
Copies the following program code to clipboard.
public struct AccountLocking()
{
    public bool EnableLockUserAccountAtRuntime { get; set; }
    public UInt32 NumberOfFailedLoginAttempts { get; set; }
    public UInt32 LoginRetryInterval { get; set; }
    public UInt32 LockAccountForDuration { get; set; }
}
private static void EnableLockingAtRuntime(DeviceItem plc)
{
    var service =
plc.GetService<PlcAccountLockingAtRuntimeFeature>();
    service.EnableLockUserAccountAtRunTime = true;
    // The number of consecutive invalid login attempts
allowed
    service.NumberOfFailedLoginAttempts = 5;
    // The duration for which incorrect login attempts are
counted
    service.LoginRetryInterval = 5;
    // The period of time for which the relevant user
```

```
    // is blocked due invalid login attempts.
    service.LockAccountForDuration = 5;
}
```

Functions for accessing PLC service 2

2.1 Access level setting

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness API to access the Access level setting via the Hardware PlcAccessLevelProvider at the Deviceitem CPU of Device.

This feature provides actions to set / reset the password and a modeled attribute to set / read the access level setting.

- void SetPassword (PlcProtectionAccessLevel accessLevelType, SecureString password)
- void ResetPassword (PlcProtectionAccessLevel accessLevelType)
- attribute PlcProtectionAccessLevel

In general the handling via Openness is similar to the TIA Portal UI.

- PlcProtectionAccessLevel can always be read and set at S7-1200/1500 PLCs.
- If access level is set to anything different than Full access (or Full access incl. fail-safe) a full access password has to be set. Otherwise there will be a compile error: "Password must not be empty"
- It is not allowed to set the same password for different access levels
- It is only possible to set / reset passwords for higher levels than the one set as access level (if PlcProtectionAccessLevel is set to HMIAccess it is only possible to set a password for ReadAccess and FullAccess)

Possible exceptions:

- EngineeringTargetInvocationException: **"Same password already exists for other access level"** - will be thrown if the user tries to set the same password for different access levels
- EngineeringTargetInvocationException: **"Cannot set password for the access level"** - will be thrown if the user tries to set a password for a stricter access level than the one selected
- EngineeringTargetInvocationException: **"Password cannot be empty"** - will be thrown if the user tries to set an empty password

- EngineeringTargetInvocationException: **"Access Level is not valid"** - will be thrown if the user tries to set the access level to FullAccessIncludingFailsafe at non failsafe PLCs

Access level

In Openness the access level is modelled as an Enum named PlcProtectionAccessLevel. The name of the Enum fields are based on the "Access Level" entries for S7-1200/1500 PLCs.

TIA UI name	Enum entry	Value	Remarks
-	None	0	For enum initialization. This value must not be set by the Openness user.
Full access (no protection)	FullAccess	1	
Read access	ReadAccess	2	
HMI access	HMIAccess	3	
No access	NoAccess	4	
Full access incl. fail-safe (no protection)	FullAccessIncludingFailsafe	5	Only available at failsafe PLCs

An EOM Attribute(Modeled Attribute) named PlcProtectionAccessLevel of type PlcProtectionAccessLevel enum is available at PLCs to Set/Get the access level.

You can use the following code example for PlcProtectionAccessLevel with No access:

```
Copies the following program code to clipboard.

DeviceItem S71500PLC = ...;
PlcAccessLevelProvider myPlcAccessLevelProvider =
S71500PLC.GetService<PlcAccessLevelProvider>();
myPlcAccessLevelProvider.PlcProtectionAccessLevel =
PlcProtectionAccessLevel.NoAccess;
```

Password

The password is implemented as SecureString for security reasons. It is only possible to set / reset the passwords for the different access levels using the corresponding action. Reading the passwords is not supported.

```
Copies the following program code to clipboard.

myPlcAccessLevelProvider.SetPassword(PlcProtectionAccessLevel.ReadAccess,
someSecureString);

myPlcAccessLevelProvider.ResetPassword(PlcProtectionAccessLevel.ReadAccess);
```

See also

Connecting to the TIA Portal
Opening a project

2.2 Accessing Software Checksum

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application
  See Opening a project

Application

You can use the Openness API to access the software checksum for a PLC station.

Program code

To access, you get the instance of PlcSoftware and load a service instance of PlcChecksumProvider.

```
Copies the following program code to clipboard.

private static void
GetChecksums(Siemens.Engineering.SW.PlcSoftware plcSoftware)

{

    Siemens.Engineering.SW.PlcChecksumProvider plcChecksumProvider
=

plcSoftware.GetService<Siemens.Engineering.SW.PlcChecksumProvider>
();

    string softwareChecksum = plcChecksumProvider.Software;
```

```
    string textListsChecksum = plcChecksumProvider.TextLists;
}
```

In case the PLC does not support the checksum calculation then the GetService call returns null.

At the PlcChecksumProvider instance the software checksum can be reached as follow:



```
Copies the following program code to clipboard.

string softwareChecksum = checksumProvider.Software;
```

The Software attribute is read only and it returns null when the program is not compiled.

See also

Connecting to the TIA Portal
Opening a project

2.3 Assigning PC interface

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open

  See Opening a Project

Application

For the interface assignment, you have to get a PcInterfaceAssignment service from Siemens.HW.Features on the device item interface.

If the PC-Station version is <= 2.0, the methods of this service will return null and interface assignment will not be possible for the interfaces that are assigned to PC-Station or Windows only or none. For the interfaces that are assigned to SW-CPU, this is again possible.

Note

The support of PC Station is very limited in the CAx interchange. Only PC Stations with version 1.0 and the default value for InterfaceAssignment are supported. The reason is these two values have impact on device structure (what is built-in, what can be plugged where). But since the parameters are not part of the AutomationML file, the values are not set during import and as a result the structure of the IPC does not fit and the other modules can not be plugged.

Program code: Interface assignment

Modify and use the following program code for interface assignment:

```
Copies the following program code to clipboard.

DeviceItem swCpu = ...;

DeviceItem myInterface = ...;

PcInterfaceAssignment provider =
myInterface.GetService<PcInterfaceAssignment>();

//user has to give device item sw cpu in case he/she wants to
assign to it. (in the future, there could be multi sw-cpu
cases)
provider.AssignInterface(PcInterfaceAssignmentMode.SoftwarePlc,
swCpu);

...

provider.AssignInterface(PcInterfaceAssignmentMode.PcStation);

...

provider.AssignInterface(PcInterfaceAssignmentMode.None);

...
```

Modify and use the following program code to get information about the current assignment mode of an interface:

```
Copies the following program code to clipboard.

PcInterfaceAssignment provider =
myInterface.GetService<PcInterfaceAssignment>();

PcInterfaceAssignmentMode mode =
provider.PcInterfaceAssignmentMode;

switch(mode)

{

case PcInterfaceAssignmentMode.None:
Console.WriteLine("Assigned to none or windows-only.");

break;

case PcInterfaceAssignmentMode.PcStation:
Console.WriteLine("Assigned to pc-station.");

break;

case PcInterfaceAssignmentMode.SoftwarePlc:
Console.WriteLine("Assigned to SoftwarePlc: " +
```

```
provider.SoftwarePlc.Name);
break;
}
```

Program code: Hardware Resource and IPC Expansion configuration

You can also select correct IPC Expansion and HardwareResource property via PcInterfaceAssignment sevice using the following code:



```
Copies the following program code to clipboard.
DeviceItem myInterface = ...;
PcInterfaceAssignment provider =
myInterface.GetService<PcInterfaceAssignment>();
//this method returns a subset of IpcExpansion values which
are available for the IPC that the interface is plugged in.
IEnumerable<string> ipcExpansionChoices =
provider.GetAvailableIPCExpansions();
string myChoice;
foreach(var choice in ipcExpansionChoices)
{
//user must select the desired value depending on his/her
configuration
if (choice.Contains("3yxx1")
{
myChoice = choice;
break;
}
}
provider.IpcExpansion = myChoice;
//After assigning IpcExpansion value, user may assign
HardwareResource value using the following enumeration.
provider.HardwareResource = HardwareResource.X101;
//OR
//myInterface.SetAttribute("HardwareResource",
HardwareResource.X101);
```

```
//myInterface.SetAttribute("IpcExpansion", mychoice);
//OR
//For Software Controllers V40.0 and upper versions, user can
use X301 and X302 slots
// to be able to configure IPC expansions
service.IpcExpansion = "6AG4133-xyyxx-xxMx";
service.HardwareResource = HardwareResource.X301;
var assignmentMode = service.PcInterfaceAssignmentMode;
Assert.AreEqual(PcInterfaceAssignmentMode.SoftwarePlc,
assignmentMode);
```

See also

Connecting to the TIA Portal
Opening a project

2.4 Setting PLC priority and resource assignment export

Application

For the resource assignment export , you have to get a Resourceassignment service from Siemens.HW.Features on the device item CPU.

If the software controller version is <30.0, the methods of this service will return null and resourceassignment export will not be possible for the cpus<V30.0. Service will only work for software controllers that has version >=V30.0

Program code: Setting PLC Priority and Resource Assignment Export

Modify and use the following program code for exporting json resource configuration with or without PLC Priority:

```
 Copies the following program code to clipboard.

DeviceItem cpu = rack. Items.Find(1);
Console.ReadKey();
Siemens. Engineering. HW. Features. ResourceAssignment
resourceAssignmentCpu =
cpu.GetService<Siemens. Engineering. HW. Features.
ResourceAssignment>( Console.WriteLine
("Set PLC PRIO = 4. Press");
Console.ReadKey();
resourceAssignmentCpu. PlcPriority = (UInt32)4;
```

```
Console.WriteLine("Get PLC PRIO: " + resourceAssignmentCpu.
PlcPriority.ToString());
try
{ Console.WriteLine("Set PLC PRIO =
5. Press"); Console.ReadKey();
resourceAssignmentCpu. PlcPriority = (UInt32)5;
}catch(Exception ex)
{} Console.WriteLine(ex.Message);
Console.WriteLine("Get PLC PRIO: " +
resourceAssignmentCpu.PlcPriority.ToString());
var resRack = rack.GetService<ResourceAssignment>();
if(resRack == null) { }
Console.WriteLine("rack resource: null");
// normal exort
Console.WriteLine("Export json: Press");
Console.ReadKey();
try
{ resourceAssignmentCpu.Export(new
FileInfo("E:\\Exports\\a.json"));
}catch(Exception ex)
{ Console.WriteLine($"{ex.Message}");
Console.WriteLine("Export success");
try
{ Console.WriteLine("Export xml:Press");
Console.ReadKey();
resourceAssignmentCpu.Export(new FileInfo("E:\\b.xml"));
}catch(Exception ex)
{ Console.WriteLine(ex.Message);
Console.WriteLine("End..");
```

2.5 Determining the status of a PLC

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Application

You can determine the state of a PLC or all PLCs in a project.

TIA Portal Openness distinguishes between the following states:

- Offline
- PLC is connected ("Connecting")
- Online
- PLC is disconnected ("Disconnecting")
- Incompatible
- Not reachable
- Protected

Program code

Modify the following program code to determine the state of a PLC:



```
public static OnlineState GetOnlineState(DeviceItem
deviceItem)
{
    OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();
    return onlineProvider.State;
}
```

Modify the following program code to determine the state of all PLCs in a project:



```
public static void
DetermineOnlineStateOfAllProjectDevices(Project project)
{
    foreach (Device device in project.Devices)
    {
        foreach (DeviceItem deviceItem in device.DeviceItems)
        {
            OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();
            if (onlineProvider != null)
            {
```

```
                OnlineState state = onlineProvider.State;
            }
        }
    }
}
```

2.6 Setting PLC Online

Introduction

You can set a currently offline PLC to an online state.

Signature

```
OnlineState OnlineProvider.GoOnline();
```

This functionality is available on the OnlineProvider service for a device item that represents a PLC as shown in the following code:



```
   DeviceItem deviceItem = ...;
OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();
OnlineState state = onlineProvider.GoOnline();
```

Note

- Attempting to set an online PLC to the online state will not have any effect on the PLC.
- The GoOnline call can take multiple (1 - 16) seconds to complete.
- If a TIA Portal transaction is active from a StartTransaction() call, the request to go online will fail.

Setting a protected PLC online

You can go online to a plc that has been loaded with a UMAC protection. This can be done by using "Legitimation for PLC with UMAC support". To subscribe to an event of online legitimation notifications you have to register your event handler on OnlineLegitimation:



```
ConnectionConfiguration onlineConfiguration = ...;
OnlineConfigurationDelegate onlineConfigurationDelegate =
OnlineCallBackMethod;
```

```
onlineConfiguration.OnlineLegitimation +=
onlineConfigurationDelegate;
...

private void OnlineCallBackMethod(OnlineConfiguration
onlineConfiguration)
{
...
```

Note

Registering for this event is only necessary if it has not yet been done in the context of Tls-communication.

Setting a PLC with custom IP address online

You can set a currently offline PLC to an online state by using a custom IP address.This is applicable when the PLC downloaded is configured with an IP address and the IP address of the offline PLC changes or uses an identical PLC with a different IP address. Now you can now go online with the Plc by entering the downloaded IP address.

Note

- This is only possible for "Plus-Plc"s (Plc1200, Plc1500,...). This mechanism is not available for Classic Plcs (Plc300, Plc400,...).
- Ensure that IP routing to the target PLC is correctly configured.

Signature

```
OnlineState OnlineProvider.GoOnline(ConfigurationAddress);
```

This functionality is available on the OnlineProvider service for a device item that represents a PLC as shown in the following code:

```
Copies the following program code to clipboard.

DeviceItem deviceItem = ...;
ConfigurationPcInterface pcInterface = ...;
ConfigurationAddress customIpAddress =
pcInterface.Addresses.Create("10.119.57.54");
OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();

OnlineState state = onlineProvider.GoOnline(customIpAddress);
```

2.7 Accessing parameters of an online connection

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

You can use the TIA Portal Openness API interface to determine or set parameters for an online connection:

- Enumerate the available connection modes to a PLC
- Enumerate the available interfaces to a PLC
- Enumerate the allocated slots
- Enumerate the available addresses of the subnets and gateways
- Set the connection parameters.

For information on IsSecureCommunication, refer Supporting secure S7 communication TLS

Program code: Determining connection parameters

Modify the following program code to enumerate the available connection modes, PC interfaces and slots:



```
public static void EnumerateConnectionModesOfPLC(DeviceItem deviceItem)
{
    OnlineProvider onlineProvider = deviceItem.GetService<OnlineProvider>();
    if (onlineProvider == null)
    {
       return; // Only cpu device items can provide OnlineProvider service
    }
    // Accessing connection configuration object
```

```
    ConnectionConfiguration configuration =
onlineProvider.Configuration;
    // Now access connection configuration members
    foreach (ConfigurationMode mode in configuration.Modes)
    {
        Console.WriteLine("Mode name:{0}", mode.Name);
        foreach (ConfigurationPcInterface pcInterface in
mode.PcInterfaces)
        {
            Console.WriteLine("PcInterface name:{0}",
pcInterface.Name);
            Console.WriteLine("PcInterface number:{0}",
pcInterface.Number);
            foreach (ConfigurationTargetInterface
targetInterface in pcInterface.TargetInterfaces)
            {
                Console.WriteLine("TargetInterface:{0}",
targetInterface.Name);
            }
        }
    }
}
```

You can also access a connection mode and a PC interface by name:

```
Copies the following program code to clipboard.
public static ConfigurationTargetInterface
GetTargetInterfaceForOnlineConnection(OnlineProvider
onlineProvider)
{
    ConnectionConfiguration configuration =
onlineProvider.Configuration;
    ConfigurationMode mode =
configuration.Modes.Find("PN/IE");
```

```
    ConfigurationPcInterface pcInterface =
mode.PcInterfaces.Find("PLCSIM", 1);
    ConfigurationTargetInterface slot =
pcInterface.TargetInterfaces.Find("2 X3");
    return slot;
}
```

Modify the following program code to enumerate the addresses of the subnets and gateways available on a PC interface:



```
Copies the following program code to clipboard.
public static void
EnumeratingPCInterfaceSubnetsAndGateways(ConfigurationPcInterface
pcInterface)
{
    foreach (ConfigurationSubnet subnet in pcInterface.Subnets)
    {
        Console.WriteLine("Subnet name:{0}", subnet.Name);
        foreach (ConfigurationGateway gateway in subnet.Gateways)
        {
            //Get the name of the gateway:
            Console.WriteLine("Gateway name:{0}", gateway.Name);
            //Get the IP address of each gateway:
            foreach (ConfigurationAddress gatewayAddress in
gateway.Addresses)
            {
                Console.WriteLine("Gateway Address:{0} has {1}",
gatewayAddress.Name, gatewayAddress.Address);
            }
        }
    }
}
```

You can also access subnets and gateways by their name or IP address:



```
Copies the following program code to clipboard.
```

```
public static void
AccessSubnetAndGatewayOfPCInterface(ConfigurationPcInterface
pcInterface)
{
    ConfigurationSubnet subnet =
pcInterface.Subnets.Find("PN/IE_1");
    ConfigurationAddress subnetAddress =
subnet.Addresses.Find("192.168.0.1");
    ConfigurationGateway gateway =
subnet.Gateways.Find("Gateway 1");
    ConfigurationAddress gatewayAddress =
gateway.Addresses.Find("192.168.0.2");
}
```

Program code: Setting connection parameters

Note

All the connection parameters previously set are overwritten when you set the connection parameters. If you have already set the connection parameters directly in the TIA Portal, it is not necessary to call `ApplyConfiguration`. If there is already an online connection to a PLC while `ApplyConfiguration` is called, an exception is thrown.

Modify the following program code to set slot parameters:



```
Copies the following program code to clipboard.
public static void SetConnectionWithSlot(OnlineProvider
onlineProvider)
{
    ConnectionConfiguration configuration =
onlineProvider.Configuration;
    ConfigurationMode mode =
configuration.Modes.Find(@"PN/IE");
    ConfigurationPcInterface pcInterface =
mode.PcInterfaces.Find("PLCSIM", 1);
    // or network pc interface that is connected to plc
```

```
    ConfigurationTargetInterface slot =
pcInterface.TargetInterfaces.Find("2 X3");
    configuration.ApplyConfiguration(slot);
    // After applying configuration, you can go online
    onlineProvider.GoOnline();
}
```

Modify the following program code to set gateway address parameters:



```
public static void
SetConnectionWithGatewayAddress(OnlineProvider
onlineProvider, string subnetName, string gatewayAddressName)
{
    ConnectionConfiguration configuration =
onlineProvider.Configuration;
    ConfigurationMode mode =
configuration.Modes.Find(@"PN/IE");
    ConfigurationPcInterface pcInterface =
mode.PcInterfaces.Find("PLCSIM", 1);
    // or network pc interface that is connected to plc
    ConfigurationSubnet subnet =
pcInterface.Subnets.Find(subnetName);
    ConfigurationAddress gatewayAddress =
subnet.Addresses.Find(gatewayAddressName);
    configuration.ApplyConfiguration(gatewayAddress);
    // After applying configuration, you can go online
    onlineProvider.GoOnline();
}
```

Modify the following program code to set subnet address parameters:



```
public static void
SetConnectionWithSubnetAddress(OnlineProvider onlineProvider,
string subnetName)
{
```

```
    ConnectionConfiguration configuration =
onlineProvider.Configuration;
    ConfigurationMode mode =
configuration.Modes.Find(@"PN/IE");
    ConfigurationPcInterface pcInterface =
mode.PcInterfaces.Find("PLCSIM", 1);
    // or network pc interface that is connected to plc
    ConfigurationSubnet subnet =
pcInterface.Subnets.Find(subnetName);
    ConfigurationAddressComposition addresses =
subnet.Addresses;
    configuration.ApplyConfiguration(addresses[0]);
    // After applying configuration, you can go online
    onlineProvider.GoOnline();
}
```

See also

Supporting secure S7 communication TLS

2.8 Accessing Configuration accessible devices

Requirement

The TIA Portal Openness application is connected to the TIA Portal.
See

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
  See Opening a project

Application

You can get a list of all available online participants on a specific interface. You can also browse the available online participants and can get information about the targets using the supported attributes.

You can only get the online participants for one specific chosen PC interface. The list of all available participants is always a snapshot and the values are valid only at the moment you receive them.

The following are the attributes of the accessible devices.

Attribute name	Type	Description
Name	String	The pnio-name of the device for Ethernet participants.
Address	String	If an IP-address was assigned it is displayed. In no IP-address was assigned the MAC-address or the DP-address is displayed.
MACAddress	String	If the device is of type Ethernet, the MAC address is shown. This attribute is only available for Ethernet participants. If the device is not of type Ethernet an empty string is returned.
DeviceSeries	String	The device series is returned. E.g "S7-1500" or "S7-400". This attribute is only available for Ethernet participants. Otherwise the string is empty.

Note

- In case of profibus devices the value of Name is always "Accessible device".
- In case of profibus devices the value of Address has always a format like "PB= [n]".

Program Code: ConfigurationAccessibleDevice

The code sample illustrates accessing the configuration accessible devices.

```
Copies the following program code to clipboard.

StationUploadProvider uploadProvider = ...;

ConnectionConfiguration configuration =
uploadProvider.Configuration;

ConfigurationMode configurationMode =
configuration.Modes.Find("PN/IE");

ConfigurationPcInterface pcInterface =
configurationMode.PcInterfaces.Find("Intel(R) Ethernet
Connection I217-LM", 1);

foreach (ConfigurationAccessibleDevice accessibleDevice in
pcInterface.GetAccessibleDevices())

{
```

```
    string name = accessibleDevice.Name;
    string address = accessibleDevice.Address;
    string macAddress = accessibleDevice.MACAddress;
    string deviceSeries = accessibleDevice.DeviceSeries;
    ConfigurationAddress configurationAddress=
pcInterface.Addresses.Create(address);
    UploadResult uploadResult =
uploadProvider.StationUpload(configurationAddress,
OnUploadConfiguration);
...
}
```

2.9 Accessing fingerprint for quick station compare

Requirement

- The Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- PLC is offline.

Application

You can use the TIA Portal Openness to get FingerprintData for different aspects of PLC device configuration which can be used to realize a quick station compare. It is done through FingerprintDataProvider service that can be acquired from a given TIA Portal. An instance of FingerprintDataProvider will be returned on GetService call, else the service will return null.

Program code: Retrieving FingerprintDataProvider from TIA Portal

```
Copies the following program code to clipboard.
TiaPortal tia = new TiaPortal(TiaPortalMode.WithUserInterface);
FingerprintDataProvider fingerprintDataProvider =
tia.GetService<FingerprintDataProvider>();
if (fingerprintDataProvider != null)
{
 ...
```

```
}
```

Parameters of FingerprintDataProvider method

To get the fingerprintData of a device, you have to call GetFingerprintData method of FingerprintDataProvider.

The following attributes are supported in FingerprintDataProvider:

Parameter name	Type	Description
configurationAddress	Siemens.Engineering.Online.Configuration	Address of device for which the fingerprintData have to be fetched.
onlineConfigurationDelegate	Siemens.Engineering.Online	Delegate that will be called to check configuration before fetching the fingerprintData.

Configuration Address

You should provide a ConfigurationAddress object to the GetFingerprintData. The address object will be used to establish a connection to the device for which the fingerprintData have to be fetched. The ConfigurationAddress object must be created in the ConnectionConfiguration of the FingerprintDataProvider. For information on IsSecureCommunication, refer Supporting secure S7 communication TLS

For example, you can use the following code to create an address object:

```
Copies the following program code to clipboard.

...
ConnectionConfiguration configuration =
fingerprintDataProvider.Configuration;
ConfigurationMode configurationMode =
configuration.Modes.Find("PN/IE");
ConfigurationPcInterface pcInterface =
configurationMode.PcInterfaces.Find("Intel(R) Ethernet Connection
```

```
I217-LM", 1);

// Create an address. This "ConfigurationAddress" is used as
parameter for getting the fingerprintData

ConfigurationAddress fingerprintAddress =
pcInterface.Addresses.Create("192.68.0.1");

...
```

The Configuration contains a list of supported Modes. You have to select one of the Modes that should be used to get fingerprints. The selected ConfigurationMode contains a list of all local PCInterfaces that support the selected Mode. You have select one of the interfaces. The desired address can be created in the Address collection of the selected ConfigurationPcInterface.

Program code: Retrieving the FingerprintData

You can start retrieving the FingerprintData of the station by calling the action GetFingerprintData. The following parameters are supported:

Parameter	Description
ConfigurationAddress	The address of the device
OnlineConfigurationDelegate	The callback to handle inhibits)

```
 Copies the following program code to clipboard.

...

OnlineConfigurationDelegate preConfigurationDelegate =
PreConfigureFingerprint;

FingerprintDataResult result =
fingerprintDataProvider.GetFingerprintData(fingerprintAddress ,
preConfigurationDelegate);

// The fingerprints

FingerprintDataItemComposition allFingerprints =
result.FingerprintDataItems;

foreach (FingerprintDataItem item in allFingerprints)

{

// do something with the fingerprint ...

}

internal void PreConfigureFingerprint(OnlineConfiguration
onlineConfiguration)
```

```
{
...
}
```

Online configuration delegate

The possible online configuration types are listed below:

Configuration name	Description and properties
OnlineConfiguration	• Base class for other configurations • OnlineConfiguration.Message: string (read only property that contains the configuration message)

Because this is the Base class for all other configurations, this property adapter is therefore available in all configurations.

OnlinePasswordConfiguration

OnlinePasswordConfiguration is derived from OnlineConfiguration. It is the base class for other configurations and has the following properties and actions.

Data Type	Type	Property/Action	Description
OnlinePasswordConfiguration	Action	Set password via method SetPassword(password:SecureString)	Enter a password to gain read access to the module.
	Property	Check for secure communication via property bool IsSecureCommunication	The property provides 'TRUE' in the case of a "Secure communication" (TLS handshake) connection to the Plc. In all other cases the attribute delivers 'FALSE

Because this is the base class for other configurations, these properties and actions are also available in its derived classes/configurations.

OnlineReadAccessPassword

The datatype of the configuration is given below:

Configuration	DataType	Description and Action
OnlinePasswordConfiguration	OnlineReadAccessPassword	Set password via SetPassword(password:SecureString) method. Enter a password to gain read access to the module

IsSecureCommunication

With this property you can be check whether the online connection to the PLC, which is used for reading of the fingerprints, is a "Secure communication" (TLS handshake) connection in order to ensure there is no man-in-the-middle who can read the passwords.

Code example for usage within the OnlineConfigurationDelegate:

 Copies the following program code to clipboard.

```
// Using base class "OnlinePasswordConfiguration"
OnlinePasswordConfiguration passwordConfiguration =
onlineConfiguration as OnlinePasswordConfiguration;
bool isSecureCommunication =
passwordConfiguration.IsSecureCommunication;
if (isSecureCommunication == true)
{
// Secure communication with (Tls handshake)
...
}

// Using derived class "OnlineReadAccessPassword"
OnlineReadAccessPassword moduleReadAccessPassword =
onlineConfiguration as OnlineReadAccessPassword;
bool isSecureCommunication =
moduleReadAccessPassword.IsSecureCommunication;
if (isSecureCommunication == true)
{
// Secure communication with (Tls handshake)
...
}
```

Unhandled configuration that can prevent accessing fingerprint causes EngineeringDelegateInvocationException and aborts action. An EngineeringDelegateInvocationException will be thrown in case of an unhandled exception within the Delegate.

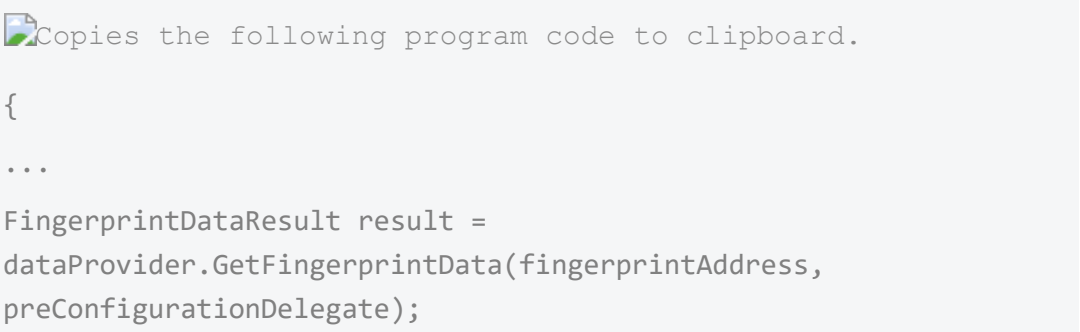
PreConfigureFingerprint implementation example



FingerprintDataResult

The FingerprintDataResult returned by the GetFingerprintData action provides a composition of all the fingerprints defined for the device.

For example, you can use the below code to examine fingerprint items:



```
FingerprintDataItemComposition allFingerprints =
result.FingerprintDataItems;
foreach (FingerprintDataItem item in allFingerprints)
{
string fingerprintDataIdentifier = item.FingerprintDataIdentifier;
string fingerprintDataValue = plcItem.FingerprintDataValue;
// Do something, e.g. store Identifier and Value in a file
}
}
```

Error Handling within the Openness fingerprints

All errors within the Openness fingerprints will be mapped as recoverable user exceptions EngineeringTargetInvocationException. So any error in an Openness action will not lead to a TIA-Portal crash.

Precondition is, that the client handle the exception. For example this could happened like shown in following code




```
TiaPortal currentTIA = ...;
FingerprintDataProvider dataProviderFingerprints =
currentTIA.GetService<FingerprintDataProvider>();
OnlineConfigurationDelegate configurationDelegate =
OnlineConfigurationFingerprintData;
try
{
// The used addr referes a PLC300.
FingerprintDataResult dataResult =
dataProviderFingerprints.GetFingerprintData(addrObj,
configurationDelegate);
// Try to get a list of all FingerprintDataItems. But a PLC300
supports no fingerprints.
FingerprintDataItemComposition dataItems =
dataResult.FingerprintDataItems;
foreach (FingerprintDataItem searchItem in dataItems)
{
```

```
string valueToCompareIdent = searchItem.FingerprintDataIdentifier;
string valueToCompareValue = searchItem.FingerprintDataValue;
DoAnything(valueToCompareIdent, valueToCompareValue);
}
}
catch (EngineeringTargetInvocationException targetException)
{
// In case of the not supported PLC this exception will be catched.
Console.WriteLine(targetException.Message);
}
```

Helper classes

For this example two classes are needed. You can use the below program code to encapsulate fingerprint data pairs.

 Copies the following program code to clipboard.

```
/// <summary>
/// Helperclass to handle a "Dictionary" as serializeable class object
/// </summary>
[Serializable]
public class FingerprintDataPairs : Dictionary<string, string>, ISerializable
{
public FingerprintDataPairs(){}
public FingerprintDataPairs(SerializationInfo si, StreamingContext context)
{
KeyValuePair<string, string>[] dataPair = si.GetValue("KeyValuePairs", typeof(KeyValuePair<string, string>[])) as KeyValuePair<string, string>[];
if (dataPair != null)
foreach (KeyValuePair<string, string> pair in dataPair)
```

```
this.Add(pair.Key, pair.Value);
}
}
```

You can use the below example to handle the fingerprint data and compare:

```
Copies the following program code to clipboard.
/// <summary>
/// Helperclass for handle the fingerprint data and compare two datasets.
/// </summary> public class FingerprintDataCompare
{
internal FingerprintDataPairs m_FingerprintDataPairs;
internal FingerprintDataPairs FingerprintDataPairSet
{
get
{
if (m_FingerprintDataPairs == null) m_FingerprintDataPairs = new FingerprintDataPairs();
return m_FingerprintDataPairs;
}
set
{
m_FingerprintDataPairs = value;
}
}
internal bool CompareDataSets(FingerprintDataPairs dataSet1, FingerprintDataPairs dataSet2)
{
foreach (string key in dataSet1.Keys)
{
if (dataSet2[key] != dataSet1[key])
return false;
```

```
}
return true;
}
internal void SaveFingerprintDataItems(FingerprintDataPairs
fingerprintDataPairSet, string filename)
{
using (FileStream fs = new FileStream(filename, FileMode.Create))
{
BinaryFormatter formatter = new BinaryFormatter();
try
{
formatter.Serialize(fs, fingerprintDataPairSet);
}
catch (SerializationException e)
{
Console.WriteLine("Failed to write fingerprints. See: " +
e.Message);
}
finally
{
fs.Close();
}
}
}
internal FingerprintDataPairs LoadFingerprintDataItems(string
filename)
{
FingerprintDataPairs dataPair = null;
using(FileStream fs = new FileStream(filename, FileMode.Open))
{
BinaryFormatter formatter = new BinaryFormatter();
try
```

```
{
dataPair = (FingerprintDataPairs)formatter.Deserialize(fs);
}
catch (System.Runtime.Serialization.SerializationException e)
{
Console.WriteLine("Failed to read fingerprints. See: " +
e.Message);
}
finally
{
fs.Close();
}
}
return dataPair;
}
}
```

Compare Fingerprints

To compare, you can use the FingerprintDataResult

```
 Copies the following program code to clipboard.
...
// See chapter "FingerprintDataResult"
FingerprintDataResult fingerprintDataResult =
dataProvider.GetFingerprintData(address,
preConfigurationDelegate);
...
```

To save the Fingerprints. you can use the Helper Classes:

```
 Copies the following program code to clipboard.
string pathDat = "FingerprintDataOverview.dat";
// Create a fingerprint comparer and fill the dataitems with the
result of "GetFingerprintData".
```

```
FingerprintDataCompare fingerprintDataSet = new FingerprintDataCompare();
foreach (FingerprintDataItem item in fingerprintDataResult.FingerprintDataItem
fingerprintDataSet.FingerprintDataPairSet.Add(item.FingerprintDataIdentifier,
item.FingerprintDataValue);
fingerprintDataSet.SaveFingerprintDataItems(fingerprintDataSet.FingerprintData
pathDat);
```

With saved FingerprintDataItems and a current FingerprintDataResult a Compare of Fingerprints can be realized

```
 Copies the following program code to clipboard.
...
string pathDat = "FingerprintDataOverview.dat";
FingerprintDataCompare fingerprintDataSet = new FingerprintDataCompare();
foreach (FingerprintDataItem item in
fingerprintDataResult.FingerprintDataItems)
fingerprintDataSet.FingerprintDataPairSet.Add(item.FingerprintDataIdentifier,
item.FingerprintDataValue);
FingerprintDataPairs loadDataSet = new FingerprintDataPairs();
loadDataSet = fingerprintDataSet.LoadFingerprintDataItems(pathDat);
if (!fingerprintDataSet.CompareDataSets(loadDataSet,
fingerprintDataSet.FingerprintDataPairSet))
{
// Here is place to implement user actions.
}
```

2.10 Accessing Cross Reference Service on Step7

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
-

Application

You can use the TIA Portal Openness to access the CrossReferenceService through GetService method on Step7 objects. The Cross Reference provides IEngineeringService i.e. CrossReferenceService on applicable Step7 objects. A null value is being returned from the TIA Portal Openness for non-applicable Step7 objects trying to access CrossReferenceService through GetService method.

The following Step7 objects that provide Cross Reference Openness support:

- OB
- FB
- FC
- DB
- InstanceDB
- GlobalDB
- Array DB
- PlcTag
- PlcSystemConstant
- Plc User Data Type

Program code

 Copies the following program code to clipboard.

```
Software softTarget = container.Software; //Accessing lower levels
until we reach the PlcBlockComposition
PlcSoftware plcSoft = (PlcSoftware)softTarget;
PlcBlockSystemGroup blockComposition = plcSoft.BlockGroup;
PlcBlockComposition blockFolder = blockComposition.Blocks;
PlcBlock mainBlock = blockComposition.Blocks.Find("Main");
if (mainBlock != null)
{
try
{
CrossReferenceService crossReferenceService =
mainBlock.GetService<CrossReferenceService>();
.....
}
catch (Exception e)
```

```
{
Console.WriteLine(e.Message);
}
}
}
```

See also

Connecting to the TIA Portal

2.11 Getting Cross References for Step7

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Introduction

You can use the TIA Portal Openness to provide Cross Reference information on applicable Step7 objects. The Openness API returns CrossReferenceResult which provide the textual values of cross reference user interface and actual IEngineeringObject of applicable Step7 objects.

The cross reference will attempt to provide the Step7 object itself on which you can perform additional actions supported by the respective TIA Portal Openness objects.

Note

Currently, the members of Step7 object does not support the type IEngineeringObject i.e. (Step7 object members - DB member, FB Members, FC Members , Tag Members, UDT members) but CrossReferenceResult includes all the textual information that is already available.

Properties

Below table provides the list of properties supported by Step7 and WinCC Unified objects in TIA Portal Openness:

Custom type	Property name	Datatype	Access Read/Write

SourceObject	Name	String	Read
	Address	String	Read
	TypeName	String	Read
	Device	String	Read
	Path	String	Read
	UnderlyingObject	Object	Read
ReferenceObject	Name	String	Read
	Address	String	Read
	TypeName	String	Read
	Device	String	Read
	Path	String	Read
	UnderlyingObject	Object	Read
Location	Name	String	Read
	ReferenceLocation	String	Read
	ReferenceType	Enum	Read
	Access	Enum	Read
	ReferencedAs	Object	Read
	ReferncedAsName	String	Read
	Address	String	Read
	TypeName	String	Read

Below table describes the list of navigators supported by Step7 and WinCC Unified objects in TIA Portal Openness:

Custom type	Navigators	Datatype

CrossReferenceResult	Sources	SourceObjectComposition
SourceObject	References	ReferenceObjectComposition
ReferenceObject	Locations	LocationComposition

Below table describes the list of methods and enums supported by Step7 and WinCC Unified objects in TIA Portal Openness:

Function Name with Signature	Enum
public CrossReferenceResult GetCrossReferences(CrossReferenceFilter filter)	CrossReferenceFilter
	ReferenceType
	Access

Below table describes the list of filter enums supported by Step7 and WinCC Unified objects in TIA Portal Openness:

Enum	Values
AllObjects	Show All Object
ObjectsWithReferences	Show Object With References
ObjectsWithoutReferences	Show Object Without References
UnusedObjects	Show Unused Objects

Program code

```
Copies the following program code to clipboard.
Public void GetCrossReferences()
{
CrossReferenceService crossReferenceService =
MainBlock.GetService<CrossReferenceService >();
CrossReferenceResult rootResultObject =
xRefService.GetCrossReferences(CrossReferenceFilter.AllObjects);
SourceObjectComposition rootResultSourceObjects =
rootResultObject.Sources; //The API returns a single source object
```

```
but the type is a composition keeping LTS in mind.
PrintSourceObjects(rootResultSourceObjects);
}
Private void PrintSourceObjects(SourceObjectComposition
sourceObjects)
{
foreach(SourceObject source in sourceObjects)
{
// In this approach, cross reference tries to provide the actual
client openness object via the "UnderlyingObject"attribute.
// However, due to the fact that cross reference result can have
an object of any type such as OB, FB, DB, Tag etc returned from
the CrossReferenceService
// and a limitation in openness that the type cannot be pre-
mapped, the user must cast the object before accessing its
attributes.
// As a result, the openness user must convert
sourceObject.UnderlyingObject to required EOM type (such as OB,
DB, FB, Tag etc)
if (source.UnderlyingObject is DataBlock)
{
var db = source.UnderlyingObject as DataBlock;//Perform actions on
db.
}
else if(source.UnderlyingObject is CodeBlock)
{
var cb = source.UnderlyingObject as CodeBlock;
//Perform actions on cb.
}
// Every new type for which support is added, the openness user
must add the casting logic as shown in the above if-else
conditional statement.
// Currently, the EOM support is NOT available for Member objects
such as Datablock member, Tag member etc.
```

```
// In such cases, the "source.UnderlyingObject" will be null.
Console.WriteLine(source.Name);
//No other attributes are available from the IEngineeringObject.
All the remaining values must be given from cross reference
separately via SourceObject EOM.
Console.WriteLine(source.Address);
Console.WriteLine(source.TypeName);
Console.WriteLine(source.Device);
Console.WriteLine(source.Path);
//Get the source object references
ReferenceObjectComposition referenceObjects =
source.References;
PrintReferences(referenceObjects);
//Get the source object's children
SourceObjectComposition sourceObjectChildren =
source.Children;
PrintSourceObjects(sourceObjectChildren);
}
}
private void PrintReferences(ReferenceObjectComposition
referenceObjects)
{
foreach (ReferenceObject referenceObject in referenceObjects)
{
// Needs similar casting for each type as seen in source object
example snippet
if (referenceObject.UnderlyingObject is DataBlock)
{
var db = referenceObject.UnderlyingObject as DataBlock;
//Perform actions on db.
}
else if(referenceObject.UnderlyingObject is CodeBlock)
{
```

```
var cb = referenceObject.UnderlyingObject as CodeBlock;
//Perform actions on cb.
}
//No other attributes are available from the IEngineeringObject.
All the remaining values must be given from cross reference
separately via ReferenceObject EOM.
Console.WriteLine(referenceObject.Name);
Console.WriteLine(referenceObject.Address);
Console.WriteLine(referenceObject.TypeName);
Console.WriteLine(referenceObject.Device);
Console.WriteLine(referenceObject.Path);
LocationComposition locations = referenceObject.Locations;
//Looping through ReferenceLocations
foreach (Location location in locations)
{
Console.WriteLine(location.Name);
Console.WriteLine(location.ReferenceLocation);
Console.WriteLine(location.ReferenceType);
Console.WriteLine(location.Access);
if(location.ReferencedAs is Tag)
var referencedAsObject = location.ReferencedAs as Tag;
}
// Needs similar casting for each type as seen in source object
example snippet
Console.WriteLine(location.ReferncedAsName);
Console.WriteLine(location.Address);
Console.WriteLine(location.TypeName);
}
}
}
```

See also

Connecting to the TIA Portal
Opening a project

2.12 Setting PLC online of R/H system

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

You can use the RHOnlineProvider service to set online either to primary PLC or backup PLCs of R/H system.

Program code: Accessing RHOnlineProvider service from a device

Modify the following code to access RHOnlineProvider:

```
 Copies the following program code to clipboard.

Device device = project.Devices.Find("S7-1500R/H-System_1");

RHOnlineProvider rhOnlineProvider =
device.GetService<RHOnlineProvider>();
```

Program code: Setting connection parameters

You can use ConnectionConfiguration object to set a connection to the device. It can be accessed from Configuration property of the RHOnlineProvider. For more information about how to set connection, Refer Accessing parameters of an online connection

Modify the following program code to set a connection mode and access a PC interface by name. For information on IsSecureCommunication, refer Supporting secure S7 communication TLS

```
 Copies the following program code to clipboard.

ConnectionConfiguration connectionConfiguration =
rhOnlineProvider.Configuration;

ConfigurationMode mode =
connectionConfiguration.Modes.Find("PN/IE");

ConfigurationPcInterface pcInterface =
mode.PcInterfaces.Find("Broadcom NetXtreme Gigabit Ethernet",
1);
```

```
ConfigurationTargetInterface targetConfiguration =
pcInterface.TargetInterfaces.Find("1 X1");

bool success =
connectionConfiguration.ApplyConfiguration(targetConfiguration);
```

Note

R/H system consists of two PLCs, a single connection configuration is provided to you.

Program code: Setting online R/H system

You can set online either to primary or backup PLC. The user attempts to set online to both targets simultaneously will encounter EngineeringTargetInvocationException from system.

Modify the following program code to set online to the primary PLC:



```
Copies the following program code to clipboard.

OnlineState onlineState =
rhOnlineProvider.GoOnlineToPrimary();
```

Modify the following program code to set online to the backup PLC:



```
Copies the following program code to clipboard.

OnlineState onlineState =
rhOnlineProvider.GoOnlineToBackup();
```

Note

You are allowed to reuse previously stored password when setting a PLC Online of R/H system.

Program code: Determining online status of R/H system

You can use PrimaryState and BackupState properties of RHOnlineProvider to determine the online connection status of primary PLC and backup PLC individually . Both properties return enum OnlineState. For more information on identify the online state of PLC, Refer Determining the status of a PLC

Modify the following program code to determine the state of primary PLC and backup PLC:



```
Copies the following program code to clipboard.

RHOnlineProvider rhOnlineProvider = ...;
```

```
OnlineState primaryState = rhOnlineProvider.PrimaryState;
OnlineState backupState = rhOnlineProvider.BackupState;
```

Program code: Setting offline R/H system

Modify the following program code to set a currently online R/H system to an offline state by invoking RHOnlineProvider.GoOffline method:

```
Copies the following program code to clipboard.
rhOnlineProvider.GoOffline();
```

Setting online on a R/H system with custom IP address

You can use a custom IP address on the RHOnlineProvider with the GoOnlineToPrimary(...) and GoOnlineToBackup(...) methods.

Signature

```
RHOnlineState OnlineProvider.GoOnlineToPrimary(ConfigurationAddress);
...
RHOnlineState OnlineProvider.GoOnlineToBackup(ConfigurationAddress);
```

See also

Accessing parameters of an online connection
Determining the status of a PLC
Connecting to the TIA Portal
Opening a project
Supporting secure S7 communication TLS

2.13 Accessing software container from primary PLC of R/H system

Requirements

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

Application

You can use primary PLC device of an R/H system to access software container, for example the R/H system will provide software container for a primary PLC device item representing PLC_1. Otherwise, it will provide null if you try to access a software container for backup PLC device representing PLC_2.

The specific of SoftwarContainer and its software property are described in Access software target.

Program code: Accessing software container

Modify the following program code to access software container from primary device of an R/H system:

```
Copies the following program code to clipboard.
foreach (DeviceItem deviceItem in rhDevice.DeviceItems)
{
  if (deviceItem.Name == "PLC_1")
  {
    SoftwareContainer softwareContainer =
deviceItem.GetService<SoftwareContainer>();
    ... //Work with softwareContainer
  }
}
```

See also

Access software target
Connecting to the TIA Portal
Opening a project

2.14 Downloading PLCs of R/H System

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project

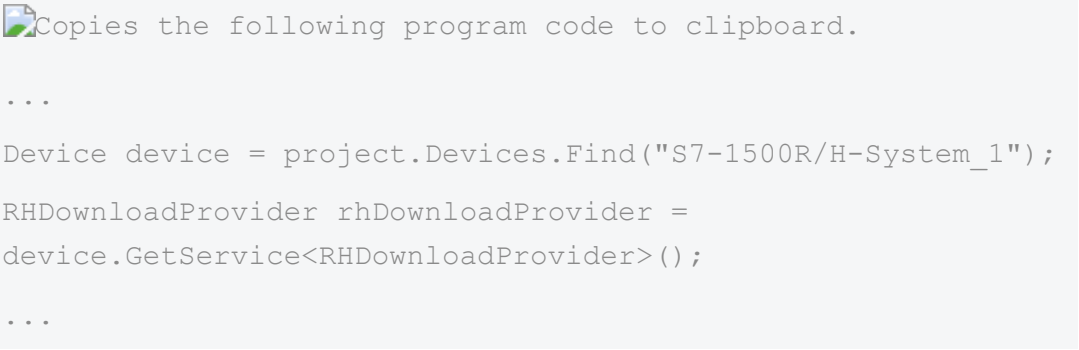
Application

You can use the TIA Portal Openness application to download the primary and backup PLCs of R/H system. You candownload both hardware and software components of the system. (Refer Downloading hardware and software components to PLC device)

Program code: Retrieving RHDownloadProvider

You can download to R/H system through RHDownloadProvider service from a device.

Modify the following program code to retrieve RHDownloadProvider:



```
...
Device device = project.Devices.Find("S7-1500R/H-System_1");
RHDownloadProvider rhDownloadProvider =
device.GetService<RHDownloadProvider>();
...
```

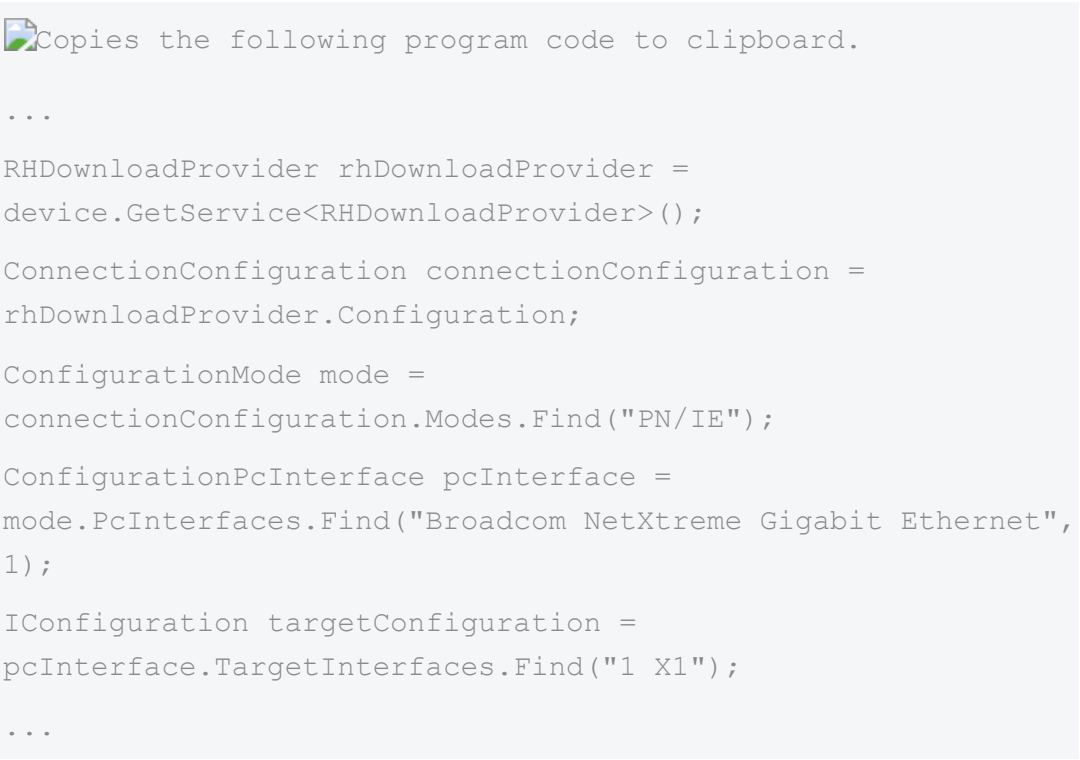
Note

The DownloadProvider service will not be accessed for CPUs that are part of R/H system.

Program code: Retrieving IConfiguration

RHDownloadProvider provides ConnectionConfiguration object through Configuration property which will be used to configuring the connection to the device. For information on IsSecureCommunication, refer Supporting secure S7 communication TLS

Modify the following program code to retrieve IConfiguration object from ConnectionConfiguration on RHDownloadProvider:



```
...
RHDownloadProvider rhDownloadProvider =
device.GetService<RHDownloadProvider>();
ConnectionConfiguration connectionConfiguration =
rhDownloadProvider.Configuration;
ConfigurationMode mode =
connectionConfiguration.Modes.Find("PN/IE");
ConfigurationPcInterface pcInterface =
mode.PcInterfaces.Find("Broadcom NetXtreme Gigabit Ethernet",
1);
IConfiguration targetConfiguration =
pcInterface.TargetInterfaces.Find("1 X1");
...
```

Note

R/H systems consist of two PLCs, only one connection configuration object is provided that can be used for both primary and backup downloads.

Program code: Downloading primary CPU and backup CPU

Modify the following program code to download to the primary CPU by invoking RHDownloadProvider.DownloadToPrimary:



```
Copies the following program code to clipboard.

DownloadResult DownloadToPrimary(configuration,
preDownloadConfigurationDelegate,
postDownloadConfigurationDelegate, downloadOptions);
```

Modify the following program code to download to the backup CPU by invoking RHDownloadProvider.DownloadToBackup:



```
Copies the following program code to clipboard.

DownloadResult DownloadToBackup(configuration,
preDownloadConfigurationDelegate,
postDownloadConfigurationDelegate, downloadOptions);
```

Parameters of RHDownloadProvider method

Both RHDownloadProvider.DownloadToPrimary and RHDownloadProvider.DownloadToBackup accept the same parameters and also return a DownloadResult. For more information about the details of IConfiguration, DownloadConfigurationDelegate, DownloadOptions and DownloadResult, Refer Downloading hardware and software components to PLC device

Parameter name	Type	Description
configuration	Siemens.Engineering.Connection.IConfiguration	Connection configuration to a device.
onlineAddress	Siemens.Engineering.Connection.ConfigurationAddress	Optional parameter for download to a changed IP address

preDownloadConfigurationDelegate	Siemens.Engineering.Download.DownloadConfigurationDelegate
postDownloadConfigurationDelegate	Siemens.Engineering.Download.DownloadConfigurationDelegate
downloadOptions	Siemens.Engineering.Download.DownloadOptions

Depending upon the state of R/H system, you might request to stop the system for the download via DownloadConfigurations. Therefore, in addition to the configuration described in Downloading hardware and software components to PLC device, the following data type are added to support RHDownload.

Configuration	Data Type	Action
DownloadSelectionConfiguration	StopHSystem	Set CurrentSelection:StopHSystemSelection Available enum values: • NoAction (No Action) • StopHSystem (Stop R/H-System)
	StopHSystemOrModule	Set CurrentSelection:StopHSystemOrModuleSel Available enum values: • NoAction (No action) • StopHSystem (Stop R/H-System) • StopModule (Stop module)
	StartBackupModules	Set CurrentSelection:StartBackupModulesSelect Available enum values: • NoAction (No action)

	• SwitchToPrimaryCpu (Change to Primary)) • StartModule (Start module)	Start modules after downloading to device
SwitchBackupToPrimary	Set CurrentSelection:SwitchBackupToPrimarySelections. Available enum values: • NoAction (No action) • SwitchToPrimaryCpu (Change to Primary)	Start modules after downloading to device.

Program code: Handling download configuration callbacks

Modify the following program code to DownloadToPrimary and DownloadToBackup invocations while handling configurations in the callbacks:

```
Download invocation example
Copies the following program code to clipboard.

static void Main(string[] args)

{

...

Project project = tiaPortal.Projects[0];

Device device = project.Devices.Find("S7-1500R/H-System_1");

RHDownloadProvider rhDownloadProvider =
device.GetService<RHDownloadProvider>();

ConnectionConfiguration connectionConfiguration =
rhDownloadProvider.Configuration;

ConfigurationMode mode =
connectionConfiguration.Modes.Find("PN/IE");

ConfigurationPcInterface pcInterface =
mode.PcInterfaces.Find("Broadcom NetXtreme Gigabit Ethernet",
1);

IConfiguration targetConfiguration =
pcInterface.TargetInterfaces.Find("1 X1");

 // Download to primary
```

```
DownloadResult primaryDownloadResult =
rhDownloadProvider.DownloadToPrimary(targetConfiguration,
PreConfigureDownloadCallback,
PostConfigureDownloadCallback,
DownloadOptions.Hardware | DownloadOptions.Software);
WriteDownloadResults(primaryDownloadResult);
// Download to backup
DownloadResult backupDownloadResult =
rhDownloadProvider.DownloadToBackup(targetConfiguration,
PreConfigureDownloadCallback,
PostConfigureDownloadCallback,
DownloadOptions.Hardware | DownloadOptions.Software);
WriteDownloadResults(backupDownloadResult);
...
}
private static void
PreConfigureDownloadCallback(DownloadConfiguration
downloadConfiguration)
{
StopHSystem stopHSystem = downloadConfiguration as
StopHSystem;
if (stopHSystem != null)
{
stopHSystem.CurrentSelection =
StopHSystemSelections.StopHSystem;
}
OverwriteTargetLanguages overwriteTargetLanguages =
downloadConfiguration as OverwriteTargetLanguages; if
(overwriteTargetLanguages != null)
{
overwriteTargetLanguages.Checked = true;
}
```

```
AlarmTextLibrariesDownload alarmTextLibraries =
downloadConfiguration as AlarmTextLibrariesDownload;
if (alarmTextLibraries != null)
{
alarmTextLibraries.CurrentSelection =
AlarmTextLibrariesDownloadSelections.ConsistentDownload;
return;
}
CheckBeforeDownload checkBeforeDownload =
downloadConfiguration as CheckBeforeDownload;
if (checkBeforeDownload != null)
{
checkBeforeDownload.Checked = true;
return;
}
ConsistentBlocksDownload consistentBlocksDownload =
downloadConfiguration as ConsistentBlocksDownload;
if (consistentBlocksDownload != null)
{
consistentBlocksDownload.CurrentSelection =
ConsistentBlocksDownloadSelections.ConsistentDownload;
return;
}
OverwriteSystemData overwriteSystenData =
downloadConfiguration as OverwriteSystemData;
if (overwriteSystenData != null)
{
overwriteSystenData.CurrentSelection =
OverwriteSystemDataSelections.Overwrite;
return;
}
}
```

```
private static void
PostConfigureDownloadCallback(DownloadConfiguration
downloadConfiguration)
{
StartModules startModules = downloadConfiguration as
StartModules;
if (startModules != null)
{
startModules.CurrentSelection =
StartModulesSelections.StartModule;
return;
}
}
private static void WriteDownloadResults(DownloadResult
result)
{
Console.WriteLine("State:" + result.State);
Console.WriteLine("Warning Count:" + result.WarningCount);
Console.WriteLine("Error Count:" + result.ErrorCount);
RecursivelyWriteMessages(result.Messages);
}
private static void
RecursivelyWriteMessages(DownloadResultMessageComposition
messages, string indent = "")
{
indent += "\t";
foreach (DownloadResultMessage message in messages)
{
Console.WriteLine(indent + "DateTime: " + message.DateTime);
Console.WriteLine(indent + "State: " + message.State);
Console.WriteLine(indent + "Message: " + message.Message);
RecursivelyWriteMessages(message.Messages, indent);
}
```

```
}
```

See also

Connecting to the TIA Portal
Opening a project
Downloading to PLC devices
Supporting secure S7 communication TLS

2.15 Establishing or disconnecting the online connection to the PLC

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- All devices are enumerated.
  See Accessing device items.

Application

You can establish the online connection to a PLC, or disconnect an existing online connection.

Program code

Modify the following program code to establish or disconnect the online connection to a PLC:

```
Copies the following program code to clipboard.
public static void SetOnlineConnection(DeviceItem deviceItem)
{
    OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();
    if (onlineProvider == null) { return; }
    // Go online
    if (onlineProvider.Configuration.IsConfigured)
    {
        onlineProvider.GoOnline();
    }
   // Go offline
```

```
   onlineProvider.GoOffline();
}
```

You can also establish or disconnect the online connections to all available PLCs in a project.

```
Copies the following program code to clipboard.
public static void SetOnlineConnectionForAllPLCs(Project
project)
{
    foreach (Device device in project.Devices)
    {
        foreach (DeviceItem deviceItem in device.DeviceItems)
        {
            OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();
            if (onlineProvider != null)
            {
                // Establish online connection to PLC:
                onlineProvider.GoOnline();
                // ...
                // Disconnect online connection to PLC:
                onlineProvider.GoOffline();
            }
        }
    }
}
```

2.16 Assigning project language to PLC

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a Project

Application

You can use the TIA Portal Openness to assign project languages to web server and display languages of an S71500 PLC. To achieve that a StructuredData type dynamic attributes is required.

To access multilingual settings the new dynamic attribute MultilingualSupport of type TableData has been added. This can be split up into Rows. At each Row the assigned project language can be set or read with the dynamic attribute ProjectLanguage.

Program code

Modify the following program code to assign project language to PLC:



```
Copies the following program code to clipboard.

//To Set the Project Language

LanguageSettings languageSettings = project.LanguageSettings;

LanguageAssociation activeLanguages =
languageSettings.ActiveLanguages;

DeviceItem Plc = …;

Tabledata multilingualSupportTable =
Plc.GetAttribute("MultilingualSupport");

StructuredDataComposition structuredDataComp =
multilingualSupportTable.Rows; // This will return a collection of
Structured Data.

StrcuturedData firstRow= structuredDataComp[0]; //First row

Language activeGermanLanguage =
activeLanguages.Find(CultureInfo.GetCultureInfo("de-DE"));

firstRow.SetAttribute("ProjectLanguage",
activeGermanLanguage.Culture);

 //To Get the Languages of device displayed

var assignedToGerman = firstRow.GetAttribute("DisplayLanguage");
```

See also

Connecting to the TIA Portal
Opening a project

2.17 Assigning watch & force tables for web server and PLC display

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a Project

Application

You can use the TIA Portal Openness to assign already created watch and force tables to the web server. The watch table and force table are software that can be found, created, and deleted in the software container. You can use the TIA Portal Openness to export/ import Watch and Force tables. For export/import Watch/Force tables, please see Export/Import Watch & Force Table

To assign the watch and force tables to the web server, the WatchAndForceTableAccessManager service is used at the PLC DeviceItem.

The service contains two navigators for watch and force tables.

- WatchtableAccessRules {WatchtableAccessRuleComposition}
- ForcetableAccessRules {ForcetableAccessRuleComposition}

The navigator WatchTableAccessRules provides the WatchTableAccessRuleComposition which contains objects of type WatchTableAccessRule. The composition is empty by default. For the WatchTableAccessRule objects, the actions Find and Create are defined.

WatchTable Access Rule & ForceTable AccessRule

- **PlcWatchTable** watchtable {get;} returns the software object watchtable. In order to exchange an assigned watch table, the user has to remove the assignment of the current one (using WatchTableAccessRule.Delete()) and add a new one (using WatchTableAccessRule.Create()).
- **WatchAndForceTableAccess** access {get; set;} returns / sets the web server access level whether it is read or read/write

TIA UI name	value	Openness enum description
-	0	None
Read	1	Read
Read/Write	2	Write

Error will be thrown in the following cases:

- If the web server has not been activated (WebserverActivate == FALSE), a EngineeringTargetInvocationException is thrown with error details stating that a WatchTableAccessRule can be added only if the web server is enabled.
- If the user tries to add a watch table with WatchAndForceTableAccess.None a ConfigOpenessUserException with error details stating that WatchAndForceTableAccess.None is not allowed as a access level

Program code: Assigning & unassigning watch table

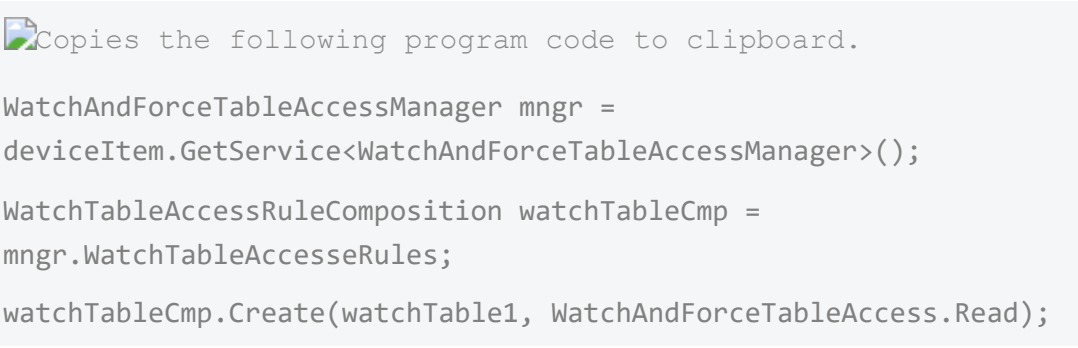
Modify the following program code to search watch table in the web server:



Note

Null will be returned if that watchtable is not associated with any WatchTableAccessRule in the Webserver

Modify the following program code to create a new watch table to the web server with read access:



Modify the following program code to delete an existing WatchTableAccessRule and unassign the watch table from the web server:



Error will be thrown in the following cases:

- If the web server has not been activated (WebserverActivate == FALSE), a EngineeringTargetInvocationException is thrown with error details stating that a WatchTableAccessRule can be added only if the web server is enabled
- If a WatchtableAccessRule with the watch table already exists, a ConfigOpenessUserException is thrown with error details stating that the watchtable already exists.
- If you try to add a watch table with WatchAndForceTableAccess.None, a ConfigOpenessUserException with error details stating that WatchAndForceTableAccess.None is not allowed as a access level.

Program code: Assigning force table to the web server

Modify the following program code to search force table in the web server:

```
WatchAndForceTableAccessManager mngr =
deviceItem.GetService<WatchAndForceTableAccessManager>();
ForceTableAccessRuleComposition forceTableCmp =
mngr.ForceTableAccessRules;
ForceTableAccess forceTable = forceTableCmp.Find(forceTable1);
```

Modify the following program code to create PLC force table to the web server with read access:

```
ForceTableAccessRuleComposition forceTableCmp =
mngr.ForceTableAccessRules;
forceTableCmp.Create(forceTable1, WatchAndForceTableAccess.Read);
```

Watch and force tables at the PLC display

You can use the same Openness service WatchAndForceTableAccessManager available at the Display submodule (DeviceItem) of the PLC DeviceItem. The same web server functionality is used as described above to assign watch and force tables at the PLC display. In contrast to the web server, the display cannot be disabled, so here a similar validation as for WebserverActive is not available.

See also

Export/Import Watch & Force Table
Connecting to the TIA Portal

2.18 Managing certificate

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to manage the following tasks with certificates:

- Creating and deleting certificate
- Exporting and importing certificate
- Assigning and unassigning certificates
- Getting certificate ID

Local certificate manager

For device certificates the service LocalCertificateManager at the DeviceItem PLC is provided. The LocalCertificateManager has a local store for the certificates named LocalCertificateStore. There the certificates for the specific PLC are stored. To get the device item that owns the instance of the service, the property OwnedBy can be used.



```
DeviceItem Plc= ...;
// Get local certificate manager
var localCertificateManager =
Plc.GetService<LocalCertificateManager>();
// Disable global certificate manager
localCertificateManager.EnableGlobalCertificatesStore =
false;
// Get local certificate store
var localCertificateStore =
localCertificateManager.LocalCertificateStore;
```

Certificate handling

Modify the following program code to create a certificate:



```
/ Create templates
```

```
var templateTls =
localCertificateStore.GetCertificateTemplate(CertificateUsage.Tls);
var templateWebserver =
localCertificateStore.GetCertificateTemplate(CertificateUsage.WebServer)
var templateOpcUaServer =
localCertificateStore.GetCertificateTemplate(CertificateUsage.OpcUaServe
...
//Template handling and configuration is handled later
// Create certificates
var certificateTls = localCertificateStore.Certificates.Create(template
var certificateWeb =
localCertificateStore.Certificates.Create(templateWebserver);
var certificateOpcUa =
localCertificateStore.Certificates.Create(templateOpcUaServer);
```

Modify the following program code to delete and export a certificate:

```
 Copies the following program code to clipboard.
var exportPath = …;
// renew a certificatecertificate
Tls.Delete();
certificateTls =
localCertificateStore.Certificates.Create(templateTlsNew);
certificateWeb.Export(new FileInfo(exportPath));
```

Note

With TIA Portal Openness V19, the Export() now accepts exportPath as a parameter and allows you to export certificate in any desired format.

Modify the following program code to assign certificate to web server and OPC UA server using dynamic attributes:

```
 Copies the following program code to clipboard.
DeviceItem opcUaSubmodule = …;
//Find assigned certificates
var foundWebCertificate =
Plc.GetAttribute("WebserverCertificate");
```

```
var foundOpcUaCertificate =
opcUaSubmodule.GetAttribute("OpcUaServerCertificate");
//Assign certificatesPlc.SetAttribute("WebserverCertificate",
certificateWeb);
opcUaSubmodule.SetAttribute("OpcUaServerCertificate",
certificateOpcUa);
//Unassign
certificatesPlc.SetAttribute("WebserverCertificate", null);
opcUaSubmodule.SetAttribute("OpcUaServerCertificate", null);
```

Modify the following program code to import certificates to the local certificate store for a PLC:

```
 Copies the following program code to clipboard.
var certificateWithoutPwd = …;
var certificateWithPwd = …;
var password = new SecureString();
//
…
// Import certificates
// Without password
var importedCertificate1 = certificates.Import(new
FileInfo(certificateWithoutPwd));
//With password
var importedCertificate2 = certificates.Import(new
FileInfo(certificateWithPwd), password);
```

Modify the following program code to get certificate ID:

```
 Copies the following program code to clipboard.
var certificateId = importedCertificate2.Id;
```

Modify the following program code to indicate if a certificate has a private key the boolean property HasPrivateKey at the certificate object can be used.

```
 Copies the following program code to clipboard.
if(importedCertificate2.HasPrivateKey)
{
//Do something
```

```
}
```

Certificate template

The templates are used as basis for creating certificates. New templates can be created at the LocalCertificateStore using the action GetCertificateTemplate(CertificateUsage). Where CertificateUsage is an enumerated with the following possible values:

- None (not supported)
- Tls
- WebServer
- OpcUaServer
- OpcUaClient
- OpcUaClientServer



```
// Create template
var certTemplate =
localCertificateStore.GetCertificateTemplate(CertificateUsage.Tls);
```

In a certificate template all properties of a certificate can be set. The access is read-write:

- Signature: The used signature algorithm of type SignatureAlgorithm which is an enumerated with the following possible values
  - None (not supported)
  - Sha1RSA
  - Sha256RSA



```
certTemplate.Signature = SignatureAlgorithm.Sha1RSA;
```

- SubjectAlternativeNames: Is a composition which contains objects of type SubjectAlternativeName (IList of type <SubjectAlternativeName>). With the action Create(SubjectAlternativeNameType, string) a new SAN can be created. The handed over parameters are the Type (of type SubjectAlternativeNameType) and the Value (of type string).
- SubjectAlternativeNameType
  - None (not supported)
  - Dns
  - Email
  - IP

- Uri

```
var san1 =
certTemplate.SubjectAlternativeNames.Create(SubjectAlternativeNameType.
"127.0.0.1");
var san2 =
certTemplate.SubjectAlternativeNames.Create(SubjectAlternativeNameType.
"192.168.178.1");
var san3 =
certTemplate.SubjectAlternativeNames.Create(SubjectAlternativeNameType.
"test@prov.com");
var san4 =
certTemplate.SubjectAlternativeNames.Create(SubjectAlternativeNameType.
"something");
san3.Delete();
```

- SubjectCommonName: string

```
string subjectCommonName = certTemplate.SubjectCommonName;
certTemplate.SubjectCommonName = "exampleSubjectCommonName";
```

Usage of type CertificateUsage

```
var usage = certTemplate.Usage;
certTemplate.Usage = CertificateUsage.OpcUaClientServer;
```

ValidFrom: Type DateTime

ValidUntil: Type DateTime

```
var validFrom = certTemplate.ValidFrom;
var validUntilDateTime = new DateTime(2080, 10, 10);
certTemplate.ValidUntil = validUntilDateTime;
```

See also

Connecting to the TIA Portal
Opening a project

2.19 Managing Web Application Configuration

Default web application configuration

You can use the DefaultWebPagesFeature service of the WebApplicationConfigurationComposition component to retrieve its details and designate any one of the applications as the default by setting the IsDefault property on the corresponding WebApplicationConfiguration object.

Property Name	Data Type	Access Read / Write
WebApplicationConfiguration	WebApplicationConfigurationComposition	Read

The service provides the following properties to modify the DefaultWebPagesFeature service.

Property Name	Data Type	Access
Name	String	Read
IsDefault	Boolean	Read/Write
ApplicationType	UInt16	Read

You can make changes to the follwoing code snippet to use the DefaultWebPagesFeature service.

```
Copies the following program code to clipboard.

public WebApplicationConfigurationComposition
WebApplicationConfiguration {get;}

var service = plc.GetService<DefaultWebPagesFeature>();

var webApp = service.WebApplicationConfiguration.Find("System
web pages");
```

Note

In a set operation for the value IsDefault, when a webserver is in deactivate state throws EngineeringNotSupportedException with the message "Web server has not been enabled."

2.20 Managing system web page

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is opening
  See Opening a project

Introduction

You can use the TIA Portal Openness to set and reset the "Enable System Web Page". This functionality is supported for S7-1500 PLCs with firmware version 4.0 or higher.

You can use the SystemWebPagesFeature service by calling GetService() method on the PLC to modify the web server system page properties.

The service uses the following properties to modify the properties of system web page:

Property name	Data type	Description	Access	Default value
EnableSystemWebPages	boolean	To enable system web pages feature with default values.	Read/Write	True

Note

During a set operation for the system web pages, if the "Enable Web server via IP address of this interface" option is disabled, an EngineeringTargetInvocationException is thrown, indicating an invalid configuration.

Program code

```
Copies the following program code to clipboard.
private static void ManagingSystemWebPage(PlcSoftware plc)
{
    var service = plc.GetService<SystemWebPagesFeature>();
    service.EnableSystemWebPages= true;
```

```
}
```

2.21 Managing Webserver user of SIWAREX modules

Requirement

- The TIA Portal is connected to the TIA Portal Openness
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to manage the following functionalities of Webserver user of SIWAREX modules:

- Activate or deactivate Webserver user
- Change name of the user
- Setting permission of the user
- Setting password of the user

You can use the SimpleWebserverUserManagement feature to get the list of Webserver user of Siwarex modules. The feature is supported for SIWAREX modules which have Webserver user.

Supported module

Currently only two modules of ET200SP family use this feature:

- 7MH4 138-6CA00-0CU0 V1.0 – TM SIWAREX WP341 HF
- 7MH4 138-6BA00-0CU0 V1.0 – TM SIWAREX WP351 HF

Property

Property name	Data type	Description
UserName	String	To set or get name of the user
Active	Boolean	To add activation of the user
Permissions	SimpleWebserverUserPermissions	To set or get permission of the user. The SimpleWebserverUserPermissions supports the following enums: • SimpleWebserverUserPermissions.ReadWrite

		• SimpleWebserverUserPermissions.ReadOnly • SimpleWebserverUserPermissions.None (if the user is deactivated) An EngineeringTargetInvocationException is thrown if you set the wrong permission
SetPassword()	Secure String	To set password of the user. The Password can only be set, if the user is active.

Program code



```
Copies the following program code to clipboard.
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.SimpleWebserverUser;
private void SettingWebserverUser()
{
//To get the list of webserver users
SimpleWebserverUserManagement feature =
subModule.GetService<SimpleWebserverUserManagement>();
// To get the specific user from the list of webserver users
SimpleWebserverUser webserverUser =
feature.WebserverUsers[1];
// To set activation of the user
webserverUser.active = true;
// To set the username of the user
webserverUser.UserName = "userName1";
// To get the username of the user
string name = webserverUser.UserName;
// To set the permissions to Read for the user
webserverUser.Permissions =
SimpleWebserverUserPermissions.ReadOnly;
SimpleWebserverUserPermissions permission =
webserverUser.Permissions;
// To set the password of the user
```

```
webserverUser.SetPassword(GetSecureString("Admin123"));
}
```

2.22 Managing Syslog configuration

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the SysLogConfigurationManager service in TIA Portal Openness to configure enable and disable Syslog feature. The feature enables you the option to select different transport protocol and configure different server for syslog.

The feature is available in S7-1500 PLC firmware version 3.1 onwards.

Property

The SysLogConfigurationManager service provides the following properties to modify the system logging configuration.

Property name	Data type	Description	Access
EnableSystemLogging	Boolean	Enable or Disable the System Logging setting of the SysLog Configuration	Read/Wr
TransportProtocol	SysLogTransportProtocolType	Specifies the constants for transport protocol options in system logging configuration. The following enum items and its corresponding value are supported: • None - 0 • TLSServerAndClientAuthentication - 1 • TLSOnlyServerAuthentication - 2 • UDP - 3	Read / Write

SysLogServerConfiguration	SysLogServerConfigurationComposition	Get or set all the server details of the system logging configuration. For details on SysLogServerConfiguration property, refer the below SysLogServerConfiguration section	Rea
SysLogAutoAcceptClient	Boolean	Enable or disable the automatic accept certificate at run time setting of the SysLog configuration	Rea
SysLogClientCertificateId	Uint	Represents client certificate Id for the System logging configuration	Rea
SysLogTrustedCertificateIds	String	Represents trusted certificate Id for the System logging configuration	Rea

SysLogServerConfiguration property

The SysLogServerConfiguration represents server configuration of the system logging configurations.

- SysLogServerConfiguration Find(string sysLogServerAddress)
- SysLogServerConfiguration Create(string sysLogServerAddress, UInt16 sysLogServerPort)

Property name	Data type	Description
SysLogServerAddress	String	Represents server address of the system logging configuration
SysLogServerPort	UInt16	Represents port number of the system logging configuration

Program code

Modify the following program code to enable or disable the system logging setting of the SysLog Configuration:

```
Copies the following program code to clipboard.
// To access EnableSysLog Property
var service = plc.GetService<SysLogConfigurationManager>();
service.EnableSystemLogging = true;
```

Modify the following program code to get or set the Transport Protocol of the system logging configuration:

```
Copies the following program code to clipboard.
var service = plc.GetService<SysLogConfigurationManager>();
service.EnableSystemLogging = true;
service.TransportProtocol =
SysLogTransportProtocolType.TLSServerAndClientAuthentication;
```

Modify the following program code to get or set all the server details of the system logging configuration:

```
Copies the following program code to clipboard.
var service = plc.GetService<SysLogConfigurationManager>();
service.EnableSystemLogging = true;
service.SysLogServerConfiguration.Create("www.test.com",
100);
```

Modify the following program code to enable or disable accept certificate at runtime setting:

```
Copies the following program code to clipboard.
var service = plc.GetService<SysLogConfigurationManager>();
service.TransportProtocol =
SysLogTransportProtocolType.TLSOnlyServerAuthentication;
plc.SetAttribute("SysLogAutoAcceptClient", true);
```

Modify the following program code to represent trusted certificate Id for the system logging configuration:

```
Copies the following program code to clipboard.
var service = plc.GetService<SysLogConfigurationManager>();
service.EnableSystemLogging = true;
```

```
service.TransportProtocol =
SysLogTransportProtocolType.TLSServerAndClientAuthentication;
plc.SetAttribute("SysLogTrustedCertificateIds",
certificate.Id.ToString());
```

Modify the following program code to represent client certificate Id for the system logging configuration:

```
var service = plc.GetService<SysLogConfigurationManager>();
service.EnableSystemLogging = true;
service.TransportProtocol =
SysLogTransportProtocolType.TLSServerAndClientAuthentication;
plc.SetAttribute("SysLogClientCertificateId", (uint)1);
```

2.23 Supporting secure S7 communication TLS

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

You can use the TIA Portal Openness to set up the parameters for an online connection.

These online connections are set up on the ConnectionConfiguration object that can be accessed via Configuration navigator of OnlineProvider service. This service exists on a DeviceItem that represents PLC.

```
DeviceItem deviceItem = ...;
OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();
ConnectionConfiguration configuration =
onlineProvider.Configuration;
```

Depending upon the available connections for a PLC, you can enumerate through the available connection modes or specifiy a certain connection mode. For each mode,you can then specify the PC Interface for that connection type.

The Find-Action of PcInterfaces has two parameters. First the name of the interfaceboard (ConfigurationPcInterface), and additional to this the number of the interfaceboard. Typically the number of interfaceboard is "1".

You can specify the slot for the interface.



```
Copies the following program code to clipboard.

foreach(ConfigurationMode mode in configuration.Modes)
{
Console.WriteLine("Mode name:{0}", mode.Name);
foreach(ConfigurationPcInterface pcInterface in
mode.PcInterfaces)
{
Console.WriteLine("PcInterface name:{0}", pcInterface.Name);
Console.WriteLine("PcInterface number:{0}",
pcInterface.Number);
foreach(ConfigurationTargetInterface targetInterface in
pcInterface.TargetInterfaces)
{
Console.WriteLine("TargetInterface:{0}",
targetInterface.Name);
}
}
}
ConfigurationMode myMode = configuration.Modes.Find("PN/IE");
ConfigurationPcInterface myPcInterface =
myMode.PcInterfaces.Find("PLCSIM", 1);
ConfigurationTargetInterface mySlot =
myPcInterface.TargetInterfaces.Find("2 X3");
```

Note

If PLCSIM is not running, it will not be included in the PcInterfaces collection.

You can also choose to set up the connection via the Addresses of a Subnet or a Gateway.



```
Copies the following program code to clipboard.

foreach(ConfigurationSubnet subnet in pcInterface.Subnets)
```

```
{
Console.WriteLine("Subnet name:{0}", subnet.Name);
foreach(ConfigurationAddress subnetAddress in
subnet.Addresses)
{
Console.WriteLine("Subnet address:{0}", subnetAddress.Name);
}
foreach(ConfigurationGateway gateway in subnet.Gateways)
{
Console.WriteLine("Gateway name:{0}", gateway.Name);
foreach(ConfigurationAddress gatewayAddress in
gateway.Addresses)
{
Console.WriteLine("Gateway address:{0}",
gatewayAddress.Name);
}
}
}
```

These settings can than be applied to online connection via the ApplyConfiguration call on the ConnectionConfiguration object. The system will return a boolean value indicating whether the configuration was set successfully.

```
 Copies the following program code to clipboard.
bool result;result = configuration.ApplyConfiguration(slot);
//or
result = configuration.ApplyConfiguration(subnetAddress);
//or
result = configuration.ApplyConfiguration(gatewayAddress);
```

After setting up the connection, you can then go online with the PLC. Each ApplyConfiguration call is independent and any subsequent call will override the previously applied settings. The settings are only valid for the current session, similar to the behavior in the UI. If the settings have already been configured in the UI, then the ApplyConfiguration call is not necessary and the online connection will use the already available settings.

If a device is already online, invoking the ApplyConfiguration will cause an exception to be thrown.

You can determine whether a PLC is already configured by using the IsConfigured property available on the ConnectionConfiguration target:



```
Copies the following program code to clipboard.

bool isConfigured = configuration.IsConfigured;
```

PLCs that only have one possible configuration will always be configured and therefore, the IsConfigured property will return true without calling ApplyConfiguration.

Note

Configuring the connection and going online will not affect the contents of the offline or online PLCs. The user will have to either download the software and/or hardware to the online PLC or upload the software from the online device in order to make the PLCs consistent.

Legacy communication

From TIA-Portal Version-17, PLCs of type S7-1500 with Firmware (FW) 2.9 communicate vis Tls.

Communication without Tls can be used by the EnableLegacyCommunication.

EnableLegacyCommunication	Description
true	Communication via Tls is enabled
false	false Communication via Tls is disabled. This type of communication is the same as in TIA-Portal Versions < V17.



```
Copies the following program code to clipboard.

DeviceItem deviceItem = ...;
OnlineProvider onlineProvider =
deviceItem.GetService<OnlineProvider>();
ConnectionConfiguration configuration =
onlineProvider.Configuration;
configuration.EnableLegacyCommunication = true;
```

Note

The security extension is only supported with PLC firmware >=V2.9 (Hardware configuration) from PLC side.

Connection parameters for communicate over Tls

With TIA-Portal V17 PLCs of type S7-1500 are available with Firmware (FW) 2.9. These PLCs support a secure communication over Tls.

The "Transport Layer Security" expects some conditions for a communication. If these condition are not fulfilled, a user decision is necessary whether to trust the communication or not.

For this a event handler can be subscribe on Tls notifications at the ConnectionConfiguration object. If a user decision is necessary, TIA Portal Openness will call this delegate and the client can handle it.

The 'ConnectionConfiguration' object can be accessed via Configuration navigator of OnlineProvider service.

Subscribe to an event of Tls-notification

To subscribe to an event of Tls-notifications you have to register your event handler on OnlineLegitimation:

```
Copies the following program code to clipboard.

private OnlineConfigurationDelegate
m_OnlineConfigurationDelegate = null;

...

ConnectionConfiguration OnlineConfiguration = ...

m_OnlineConfigurationDelegate = OnlineCallBackMethod;

OnlineConfiguration.OnlineLegitimation +=
m_OnlineConfigurationDelegate;
```

Unsubscribe to an event of Tls-notification

To unsubscribe to an event of Tls-notifications (OnlineLegitimation) you have to register your event handler:

```
Copies the following program code to clipboard.

ConnectionConfiguration OnlineConfiguration = ...

m_OnlineConfigurationDelegate = OnlineCallBackMethod;

OnlineConfiguration.OnlineLegitimation -=
m_OnlineConfigurationDelegate;
```

TlsVerificationConfiguration

When going online to a Plc1500 with a FW 2.9 or higher, a user decision may be necessary as to whether the connection should be established at the given circumstances. In this case the registered event handler delegate is called in the client program. For the given case, the OnlineConfiguration parameter must be checked for "TlsVerificationConfiguration".

The possible values of TlsVerificationConfigurationSelection are:

TlsVerificationConfigurationSelection	Description
NonVerified	The connection is not verified and not trusted
Trusted	The connection is verified and trusted
NonTrusted	The connection is verified and not trusted



```
private void OnlineCallBackMethod(OnlineConfiguration
onlineConfiguration)
{
TlsVerificationConfiguration verificationConfiguration =
onlineConfiguration as TlsVerificationConfiguration;
if (verificationConfiguration != null)
{
verificationConfiguration.CurrentSelection =
TlsVerificationConfigurationSelection.Trusted;
//To establish a connection the value Trusted is necessary.
}
}
```

With all other values a go online is not possibly and leads to an user exception.

Also TlsVerificationConfiguration supports two attributes:

Name of method	Description
PlcName	The name of the PLC calling the delegate.
VerificationInfo	Message for the user. Contains reason for calling the delegate.

See also

Connecting to the TIA Portal
Opening a project

2.24 Updating module description

Requirement

- The TIA Portal Openness is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Application

In TIA Portal Openness, a specific service ModuleDescriptionUpdater is used at the device item to update the current module description to the newest version of the module. To get the device item that owns the instance of the service, the property OwnedBy can be used.

You can use the CanUpdate attribute to show if a new ConfigObject version for this deviceitem is available.

Atribute	Data type	Description
CanUpdate	bool	TRUE: A new version is available FALSE: There is no new version

You can use the action UpdateModuleDescription ( ) to update the ConfigObject version of a deviceitem.

Action	Return value	Description
UpdateModuleDescription	True	The deviceitem is up to date: • The deviceitem was updated successfully • The deviceitem was already up to date
	False	The deviceitem is not up to date: • The deviceitem could not be updated

Program code

```
Copies the following program code to clipboard.
DeviceItem deviceItem = ...;
var descriptionUpdater =
deviceItem.GetService<ModuleDescriptionUpdater>();
if (descriptionUpdater != null)
{
if (descriptionUpdater.CanUpdate) //e.g is update module version
possible
{
bool result = descriptionUpdater.UpdateModuleDescription();
.
.
}
}
```

See also

Connecting to the TIA Portal
Opening a project

2.25 Supporting UMAC on PLC device

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to perform actions on PLC device protected via User Management Access Control (UMAC).

A legitimation via the event handler OnlineLegitimation is required for all online PLC functions that require legitimation and which support UMAC. This is the same event handler that is used for communication via TLS. For information about OnlineLegitimation event handler, see Subscribe to an event of Tls-notification section of Supporting secure S7 communication TLS

To register the event handler on online legitimation,you need the connection configuration object. You can access the Connection Configuration object via Configuration navigator of the corresponding online service (e.g. OnlineProvider, FingerprintDataProvider, DownloadProvider, StationUploadProvider, etc.).

Program code: Accessing connection configuration object

To access the configuration object modify the following program code:

```
 Copies the following program code to clipboard.

...
DeviceItem deviceItem = ...;
DownloadProvider downloadProvider =
deviceItem.GetService<DownloadProvider>();
ConnectionConfiguration onlineConfiguration =
downloadProvider.Configuration;
...
```

Program code: Subscribe to an event for online legitimation

To subscribe to an event of online legitimation notifications you have to register your event handler on OnlineLegitimation:

```
 Copies the following program code to clipboard.

ConnectionConfiguration onlineConfiguration = ...;
OnlineConfigurationDelegate onlineConfigurationDelegate =
OnlineCallBackMethod;
onlineConfiguration.OnlineLegitimation +=
onlineConfigurationDelegate;
...
private void OnlineCallBackMethod(OnlineConfiguration
onlineConfiguration)
{
...
```

Note

Registering for this event is only necessary if it has not been done in the context of Tls-communication.

Program code: Unsubscribe to an event for online legitimation

To unsubscribe to an event of online notifications (OnlineLegitimation) you have to unregister your event handler:



```
Copies the following program code to clipboard.

ConnectionConfiguration onlineConfiguration = ...;

OnlineConfigurationDelegate onlineConfigurationDelegate =
OnlineCallBackMethod;

onlineConfiguration.OnlineLegitimation -=
onlineConfigurationDelegate;

...

private void OnlineCallBackMethod(OnlineConfiguration
onlineConfiguration)

{

..
```

OnlineAuthenticationConfiguration

The OnlineAuthenticationConfiguration has the following properties and methods:

OnlineAuthenticationConfiguration	Description
bool IsSecureCommunication { get; }	The property provides 'true' in the case of a "Secure communication" (TLS handshake) connection to the PLC. In all other cases the attribute delivers 'false'. For more information, see Downloading to PLC devices
IList<AuthenticationType> GetSupportedAuthenticationTypes()	Get all supported AuthenticationTypes via GetSupportedAuthenticationTypes() from the PLC.
OnlineCredentials OnlineCredentials { get; }	Property to configure the credential details.

IsSecureCommunication

In the OnlineAuthenticationConfiguration object, You can verify whether the connection is using secure Tls by using the IsSecureCommunication option. This is similar to the configuration for

upload or download through the DownloadConfiguration.

```
Copies the following program code to clipboard.

OnlineAuthenticationConfiguration
onlineAuthenticationConfiguration = onlineConfiguration as
OnlineAuthenticationConfiguration;

if (onlineAuthenticationConfiguration != null)

{

    // Secure communication via TLS?

    if
(onlineAuthenticationConfiguration.IsSecureCommunication)

    {

      ...

    }

}
```

GetSupportedAuthenticationTypes() and AuthenticationType

GetSupportedAuthenticationTypes()

The different authentication legitimation types are possible depending on the downloaded UMAC configuration. To get the supported types you have to call:

```
Copies the following program code to clipboard.

OnlineAuthenticationConfiguration onlineAuthenticationConfiguration
= onlineConfiguration as OnlineAuthenticationConfiguration;

if (onlineAuthenticationConfiguration != null)

{

    IList<AuthenticationType> supportedTypes =
onlineAuthenticationConfiguration.GetSupportedAuthenticationTypes();

    foreach (AuthenticationType supportedType in supportedTypes)

    {

       ...

    }

}
```

AuthenticationType

The AuthenticationType represents the type of authentication that you can choose.
To get the type of authentication, you can get the UserType value of the UserType at the CurrentUserType attribute.

AuthenticationType	Attribute	Description
UserType CurrentUserType { get; }	Deliver the UserType, supported by the PLC	This value is usable in OnlineCredentials: UserType The possible UserType entries for Legitimation supported by the PLC are: • None - No legitimation is required • AnonymousUser - Legitimation as an anonymous user • GlobalUser - Legitimation as a global user • ProjectUser - Legitimation as a project user • SingleSignOnUser - Single sign-on user • PasswordOnly - Legacy legitimation with PLC protection access level password only



```
...
IList<AuthenticationType> supportedTypes =
onlineAuthenticationConfiguration.GetSupportedAuthenticationTypes();
foreach (AuthenticationType supportedType in supportedTypes)
{
    UserType userType = supportedType.CurrentUserType;
    if (userType == UserType.ProjectUser)
    {
        ...
    }
}
```

```
...
```

OnlineCredentials

The OnlineCredentials has the following properties and methods:

OnlineCredentials	Action	Description
void SetPassword(SecureString password)	Set the password via SetPassword method.	Enter a password into the OnlineCredentials to gain access to the module.
string Name { get; set; }	Get or set the UserName via Name property.	Enter name as UserName into the OnlineCredentials.
UserType Type { get; set; }	Get or set the UserType via Type poperty	Enter UserType into the OnlineCredentials. The possibly values are available via GetSupportedAuthenticationTypes()

After you verify the supported user types and necessary legitimation informations, you have to set these within the OnlineCredentials:



```
Copies the following program code to clipboard.

OnlineAuthenticationConfiguration
onlineAuthenticationConfiguration = onlineConfiguration as
OnlineAuthenticationConfiguration;

if (onlineAuthenticationConfiguration != null)

{

   // Check supported user types and legitimation information

   ...

   // Set the online credentials for legitimation

   OnlineCredentials onlineCredentials =
onlineAuthenticationConfiguration.OnlineCredentials;

   SecureString password = ...;

   onlineCredentials.Type = UserType.ProjectUser;

   onlineCredentials.Name = "User1";

   onlineCredentials.SetPassword(password);
```

```
}
```

Note

The explicit legitimation is not required within the callback. The legitimation will be done after the callback return. In case of insufficient access rights, the online action (e.g. download) will abort with an exception of an illegitimate connection.

2.26 Supporting IP Accessibility

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the TIA Portal Openness to support IP accessibility. In TIA Portal Openness, a new dynamic attribute PlcAccessCommunicationModule gets available at the device item CPU. The attribute is of type object, so that you can access to the CP object you want to select for IP Accessibility and assign the object to PlcAccessCommunicationModule.

If a CP has already been selected you can retrieve the CP object by getting the value of PlcAccessCommunicationModule.

Program code



```
Copies the following program code to clipboard.

DeviceItem Plc = ...;

object communicationModule =
Plc.GetAttribute("PlcAccessCommunicationModule");

// cP contains the object reference of a plugged CP

Plc.SetAttribute("PlcAccessCommunicationModule", cP);
```

See also

Connecting to the TIA Portal
Opening a project

2.27 Accessing PLC access control configuration provider

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open
  See Opening a project

Introduction

You can use the PlcAccessControlConfigurationProvider service in TIA Portal Openness to enable and disable the access control configuration at run time.

With TIA Portal Openness v20, you can use the PlcAccessControlConfigurationProvider service in TIA Portal Openness to download central users and UMC configurations.

Note

With TIA Portal Openness v20, the the PlcAccessControlConfigurationProvider service is extened with four new attributes and enums to set the local user and center user management.

Method, Parameter, and Enum

Method

The PlcAccessControlConfigurationProvider service provides an attribute called PlcAccessControlConfiguration, which supports the following functions to configure the access control configuration.

Method name	Return Datatype	Description
GetAttribute(string attributeName)	integer	Used to get PlcAccessControlConfiguration
SetAttribute(string attributeName, int configuration)	void	Used to set PlcAccessControlConfiguration

Note

All S7-1500 PLCs having firmware version 3.1 and above supports PlcAccessControlConfiguration.

Parameter

The GetAttribute() method accepts the following parameter to get the local user and central user management :

Parameter name	Data type	Description
attributeName	string	Specifies the name of the parameter, for example "PlcAccessControlConfiguration" in this case. The possible return Enum items are: • Disabled • EnabledWithAccessControl • EnabledWithAccessControlViaAccessLevel

The SetAttribute() method accepts the following parameter to set the local user and central user management:

Parameter name	Data type	Description
attributeName	string	Specifies the name of the parameter, for example "PlcAccessControlConfiguration" in this case
configuration	The possible return Enum items are: • Disabled • EnabledWithAccessControl • EnabledWithAccessControlViaAccessLevel	Valid Plc Access Control Configuration. It returns void for invalid scenarios, which will be handled by the exception.
UMC server address*	string	To connect the UMC server in PLC to fetch global users
UMC server id*	string	A certificate fingerprint which can be used to authenticate the UMC Server .
UMC Connection State*	UmcConnectionType	Specifies the connection type of UMC
UMC Trusted Certificate Id*	string	Specifies the certificate id of UMC trust.

The UmcConnectionTechnique accepts the following enum values:

Enum item	Value	Description
VerifyConnectionUsingServerId	1	User verify the connection by using the Server Id.
VerifyConnectionUsingCertificate	2	User verify the connection by using Certificate

Note

The parameter name with * is supported in all S71500PLCs having version 4.0 and above.

Enum

The PlcAccessControlConfiguration provides the following Enum values and its description:

Enum	Value	Description
Disabled	0	No change in accessing the PLC through Automated System.
EnabledWithAccessControl	2	There is no separate configuration for different modes of accessing the PLC like (AccessLevel,WebServer,OPCUA).
EnabledWithAccessControlViaAccessLevel	3	There no separate configuration for different modes of accessing the PLC like (WebServer,OPCUA); however, you can access the PLC through access level passwords.
EnabledWithAccessControlAndCentralUser*	4	There is no separate configuration for different modes of accessing the PLC like (WebServer,OPCUA); however, you can access the PLC through access level passwords.

EnabledWithAccessControlViaAccessLevelAndCentralUsers*	5	There is no separate configuration for different modes of accessing the PLC like (WebServer,OPCUA); however, you can access the PLC through access level passwords..
EnabledWithAccessControlAndLocalWithCentralUsers*	6	There is no separate configuration for different modes of accessing the PLC like (WebServer,OPCUA); however, you can access the PLC through access level passwords.
EnabledWithAccessControlViaAccessLevelAndLocalWithCentralUsers*	7	There is no separate configuration for different modes of accessing the PLC like (WebServer,OPCUA); however, you can access the PLC through access level passwords..

Note

The Enum name with ***** is supported in all S7 1500 PLCs having version 4.0 and above.

Program code

To get attribute modify the following program code:

```
// Example to use 'GetAttribute'
```

```
PlcAccessControlConfigurationProvider provider =
plc.GetService<PlcAccessControlConfigurationProvider>();
var configuration =
provider.GetAttribute("PlcAccessControlConfiguration");
```

To set attribute modify the following program code:

```
 Copies the following program code to clipboard.
// Example to use 'SetAttribute'
PlcAccessControlConfigurationProvider provider =
plc.GetService<PlcAccessControlConfigurationProvider>();
provider.SetAttribute("PlcAccessControlConfiguration",
PlcAccessControlConfiguration.EnabledWithAccessControlViaAccessLevel);
```

Program code

To access attributes UmcServerAddress from client side modify the following program code:

```
 Copies the following program code to clipboard.
private static void AccessUmcServerAddress(PlcSoftware plc)
{
   // 1. Example to use 'GetAttribute'
   var provider =
plc.GetService<PlcAccessControlConfigurationProvider>;
   var configuration =
provider.GetAttribute("UmcServerAddress");
   // 2. Example to use 'SetAttribute'
   PlcAccessControlConfigurationProvider provider =
plc.GetService<PlcAccessControlConfigurationProvider>();
   provider.SetAttribute("UmcServerAddress", attributeValue);
}
```

To access attributes UmcServerID from client side modify the following program code:

```
 Copies the following program code to clipboard.
private static void AccessUmcServerID(PlcSoftware plc)
{
    // 1. Example to use 'GetAttribute'
```

```
    var provider =
plc.GetService<PlcAccessControlConfigurationProvider>;
    var configuration = provider.GetAttribute("UmcServerId");
    // 2. Example to use 'SetAttribute'
    PlcAccessControlConfigurationProvider provider =
plc.GetService<PlcAccessControlConfigurationProvider>();
    provider.SetAttribute("UmcServerId", attributeValue);
}
```

To access attributes UmcConnectionState from client side modify the following program code:



```
Copies the following program code to clipboard.
private static void AccessUMCConnectionState(PlcSoftware plc)
{
    // 1. Example to use 'GetAttribute'
    var provider =
plc.GetService<PlcAccessControlConfigurationProvider>;
    var configuration =
provider.GetAttribute("UmcConnectionState");
    // 2. Example to use 'SetAttribute'
    PlcAccessControlConfigurationProvider provider =
plc.GetService
    <PlcAccessControlConfigurationProvider>();
    provider.SetAttribute("UmcConnectionState",
UmcConnectionTechnique.VerifyConnectionUsingCertificate);
}
```

To access attributes UmcTrustedCertificateIds from client side modify the following program code:



```
Copies the following program code to clipboard.
private static void AccessUMCTrustedCertificateID(PlcSoftware
plc)
{
    // 1. Example to use 'GetAttribute'
    var provider =
plc.GetService<PlcAccessControlConfigurationProvider>;
```

```
    var configuration =
provider.GetAttribute("UmcTrustedCertificateIds").

    // 2. Example to use 'SetAttribute'

    PlcAccessControlConfigurationProvider provider =
plc.GetService

    <PlcAccessControlConfigurationProvider>();

    provider.SetAttribute("UmcTrustedCertificateIds",
attributeValue);

}
```

2.28 Setting a display password

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a Project

Application

You can use the TIA Portal Openness to set the display protection password. You can use the method SetPassword(SecureString password) to set the display protection password where the parameter for the password to be a SecureString. The confirmation password is not needed in Openness and therefore not available.

The feature will be available if the following preconditions are met.

- The PLC has a “Display” instance.
- The “Display” supports “Password” protection feature. There are cases eg. Software PLC where Display is available but no password protection available.

Program code

Modify the following program code to get the device item that owns the instance of the service, the property OwnedBy can be used.

```
Copies the following program code to clipboard.

DisplayProtection  displayProtection  =
plcDisplay.GetService<DisplayProtection>();

var displayDeviceItem = displayProtection.OwnedBy;
```

Modify the following program code to set the Display Protection password:

```
Copies the following program code to clipboard.
displayProtection.SetPassword(someSecuredString);
//Sets the string someSecuredString as the CPU display protection
password.
```

Note

SW controllers have a display submodule, but do not support any display protection. So in TIA Portal Openness the feature SetPassword is not available at the display submodule of any SW controllers.

See also

Connecting to the TIA Portal
Opening a project

2.29 Accessing web server and OPC UA user management

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
  See Connecting to the TIA Portal
- A project is open
  See Opening a Project

Application

You can use the TIA Portal Openness to access web server and OPC UA submodule of PLCs. It is possible to add a user up to maximum number at the web server and the OPC UA submodule of PLCs. The user has a user name and a password.

User management at Web Server

You can perform the following operations, such as adding user, deleting user, and setting password at web server for all supported device only if web server is activated at the module.

If the web sever is not activated, the add, delete and set operations will throw a EngineeringTargetInvocationException, but still the read operations will be available.

The WebServerUserManagement service will be available for the PLC instance of only the supported devices. For other non-supported devices, the service will be null.

```
Copies the following program code to clipboard.
WebServerUserManagement
```

```
{
Navigators WebServerUsers
{
WebServerUserComposition
}
}
```

The Service provides a navigator called WebServerUsers using which a WebServerUserComposition can be obtained, which manages the WebServerUser instances.

```
 Copies the following program code to clipboard.
WebServerUserComposition
{
WebServerUser Find(string username);
void Create(string username, WebserverUserPermissions
permissions,SecureString password);
}
WebServerUser
{
string UserName{get;}
WebserverUserPermissions Permissions {get;set;}
void Delete();
void SetPassword(SecureString password);
}
```

Program code: Actions at WebServerUserComposition

Modify the following program to search web server user:

```
 Copies the following program code to clipboard.
WebServerUserComposition webServerUserComposition =
WebServerUserManagement.WebServerUsers;
WebServerUser user1 = webServerUserComposition.Find("user1");
```

Modify the following program code to create a new web server user with a web server user permissions:

```
Copies the following program code to clipboard.
WebServerUserComposition webServerUserComposition =
WebServerUserManagement.WebServerUsers;
WebServerUser user1 = webServerUserComposition.Create("user1",
WebserverUserPermissions.ReadTagStatus, someSecureString);
```

Modify the following program code to delete the web server user:

```
Copies the following program code to clipboard.
WebServerUserComposition webServerUserComposition =
WebServerUserManagement.WebServerUsers;
WebServerUser user1 = webServerUserComposition.Find("user1");
user1.Delete();
```

Modify the following program code to replace the current password of the user with the new password:

```
Copies the following program code to clipboard.
WebServerUserComposition webServerUserComposition =
WebServerUserManagement.WebServerUsers;
WebServerUser user1 = webServerUserComposition.Find("user1");
user1.SetPassword(someSecureString);
```

User Management at OPC UA

You can perform the perform following operations such as adding user, deleting user, and setting password at the OPC UA Server only if the OPC UA Server is activated and username and password authentication is enabled.

If these conditions are not met, the operation throws a EngineeringTargetInvocationException with error details stating that the authentication needs to be enabled.

The OpcUaUserManagement service will be available on the OPC UA submodule of the PLC. The Service will be available for all supported devices where OPC UA submodule is available for the device item. In other cases, the service will be returned as null.

```
Copies the following program code to clipboard.
OpcUaUserManagement
{
Navigators OpcUaUsers
{
```

```
OpcUaUserComposition
}
}
```

The Service provides a navigator called OpcUaUsers using which a OpcUaUserComposition can be obtained. The composition manages the OpcUaUser instances:

```
Copies the following program code to clipboard.
OpcUaUserComposition
{
OpcUaUser Find(string username);
void Create(string username, SecureString password);
}
OpcUaUser
{
string UserName{get;}
void Delete();
void SetPassword(SecureString password);
}
```

Program code: Actions at OpcUaUserComposition

Modify the following program code to search a OpcUaUser:

```
Copies the following program code to clipboard.
OpcUaUserComposition opcUaUserComposition =
opcUaUserManagement.OpcUaUsers;
OpcUaUser user1 = opcUaUserComposition.Find("user1");
```

Modify the following program code to create a new OpcUaUser:

```
Copies the following program code to clipboard.
OpcUaUserComposition opcUaUserComposition =
opcUaUserManagement.OpcUaUsers;
opcUaUserComposition.Create("user1", someSecureString);
```

Modify the following program code to delete the OpcUaUser:

```
Copies the following program code to clipboard.
OpcUaUserComposition opcUaUserComposition =
opcUaUserManagement.OpcUaUsers;
OpcUaUser user = opcUaUserComposition.Find("user1");
user.Delete();
```

Modify and use the program code to replace the current password with a supplied one:

```
Copies the following program code to clipboard.
OpcUaUserComposition opcUaUserComposition =
opcUaUserManagement.OpcUaUsers;
OpcUaUser user = opcUaUserComposition.Find("user1");
user.SetPassword(someSecureString);
```

See also

Connecting to the TIA Portal
Opening a project